



Technische Universität Berlin

Chair of Database Systems and Information Management

Master's Thesis

Hardware-Conscious Performance Engineering for Distribution-Aware Dataset Search

**A Re-Examination of Fainder for Hardware-Near
Optimisation**

Tarik Abu Mukh, t.abu.mukh@campus.tu-berlin.de

Degree: M.Sc. Computer Science

Matriculation Number: 0501953

Reviewers

Prof. Dr. Volker Markl

Prof. Dr. Matthias Böhm

Advisor(s)

Lennart Behme

Submission Date

July 24, 2026

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such. Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023.

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, July 24, 2026

.....
Tarik Abu Mukh

Documentation of Tool Usage

In accordance with the requirements for good scientific practice, below I have documented my use of AI-based tools in the process of producing this thesis. The documentation follows the work-phase-oriented format of Baresel et al., *KI-Gebrauch im Studienkontext dokumentieren* [8], including its four-level scale for the degree of use:

Degree	Characterisation
1	For inspiration (suggestions considered, own formulation)
2	Supplementary (explanations, summaries, formatting suggestions)
3	Supportive (dialogic drafting, AI output iteratively revised by the author)
4	Content-shaping (generated content adopted directly, e.g. program code)

Tools used: **Claude Code** (Anthropic, Claude model family) and **GitHub Copilot** (Microsoft). For degrees 3–4, the evidence column points to the artefact repository (<https://github.com/tmukh/Fainder-Redone>), where the generated code is archived.

Process phase	Degree	Tool and example/evidence	Reflection
Finding and defining the topic	—	—	—
Literature research	—	—	—
Implementing the prototype	3	Claude Code: boilerplate scaffolding (Cargo file setup, PyO3 binding stub boilerplate, testing boilerplate code)	Engine design, algorithms, and optimisation choices are my own.
Data analysis	3	Claude Code: SQL query generation over the measurement database, analysis code in <code>analysis/</code> . overview of perf capabilities for sweeps to see what data can be collected	All interpretations and mechanism explanations are my own, generated queries fine-tuned to extract the data we needed
Revising (content, structure, language)	2	Claude Code: consistency checks of numerical claims against the measurement database (bench.db), data styling (tabular vs in-text). Help with $\LaTeX$ Errors. matplotlib and TikZ code generation, regeneration scripts in the artefact	factual checks traced every number to a database row. Figure content and design intent specified by me, code adopted after review, data is my own
Bibliography and formatting	3	Claude Code: BibTeX and $\LaTeX$ formatting	Reference selection is my own
Translation (German abstract)	2	DeepL: translation of technical terminology and selected phrasings from the English abstract	Final wording revised by me; both abstracts state identical claims
Spell checking	1	Grammarly: spelling and grammar checks on the thesis text	Corrections reviewed and applied by me

All scientific claims, including the four-ceiling characterisation, negative composability, pre-flight ceiling identification, and the dispatch policy, are my own work.

All methodology choices, formulations of hypotheses, and interpretations of the empirical results are my own work. AI-generated output was used for drafting, formatting, and consistency checks, but not for content generation. I bear sole responsibility for the content of this thesis, and for any errors or omissions that may remain.

Zusammenfassung

Verteilungsbasierte Datensatzsuche adressiert die Aufgabe, Datensätze anhand ihrer statistischen Eigenschaften zu finden.¹ In Umgebungen, in denen die Rohdaten für die Suchenden nicht zugänglich sind (z. B. aus Datenschutz- oder Speichergründen), stehen nur spaltenweise Histogramme zur Verfügung. Fainder [9] ist ein Index für dieses Problem: er clustert Histogramme mit K-Means, richtet sie innerhalb jedes Clusters auf ein gemeinsames Bin-Gitter aus und berechnet kumulative Dichten vor, sodass Perzentilprädikate durch Binärsuche statt durch Histogrammdurchläufe beantwortet werden. Auf der GitTables-Kollektion realer Histogramme erreicht Fainder eine Beschleunigung von mehr als zwei Größenordnungen gegenüber der `profile-scan`-Baseline und reduziert den Speicherbedarf von über 1 TB auf 3,2 GB. Die Referenzimplementierung ist jedoch in Python geschrieben, und der Python-Overhead dominiert ihre Abfragezeit: der Global Interpreter Lock serialisiert die Cluster-Schleife über Threads hinweg, das Array-of-Structs-Layout von NumPy verschmutzt Cache-Lines mit ungelesenen Histogramm-ID-Feldern, und der Interpreter-Dispatch pro Iteration verstärkt beides. Das Ergebnis ist eine Laufzeit von 2,5 Stunden für 10 000 Abfragen auf dem vollen Workload mit fünf Millionen Histogrammen, unabhängig von der Thread-Anzahl.

Diese Arbeit untersucht, wie Fainders Abfragephase auf moderner Mehrkern-Hardware effizient implementiert werden kann; dafür reimplementieren wir sie in Rust. Wir finden, dass die Abfrageleistung durch vier verschiedene Hardware-Obergrenzen begrenzt ist: Ein-Thread-Ladelatenz, gemeinsame Cache-Kapazität, Speicherverkehrsdruck innerhalb eines Prozessorsockels und Kommunikation zwischen den Sockeln. Jede Obergrenze bindet bei einer anderen Thread-Anzahl und wird von einer eigenen Klasse von Interventionen adressiert. Wir finden außerdem, dass Optimierungen, die diese Obergrenzen adressieren, *negativ* komponieren. Über fünf Optimierungspaare wiederholt sich dasselbe Muster: eine gröbere Optimierung hat den Spielraum an der Ziel-Obergrenze bereits aufgebraucht, und der Overhead der feineren Optimierung wird zu einer Netto-Regression statt zu einem abnehmenden Ertrag.

¹Fachbegriffe und ausgewählte Formulierungen dieser Zusammenfassung wurden mit *DeepL* (<https://www.deepl.com/>) übersetzt, um Terminologie und Satzbau des englischen Abstracts zu bewahren.

Die neu entwickelte Engine beantwortet Abfragen Ende-zu-Ende $530\text{--}598\times$ schneller als die Python-Baseline bei exakt identischen Ergebnissen. Der Großteil dieses Verhältnisses stammt daraus, dass Rust Pythons Materialisierungskosten pro Ergebnis vermeidet, nicht aus schnellerer Suche. Die Suchphase selbst ist auf kleinen Workloads bis zu $22,7\times$ schneller und nähert sich mit wachsendem Workload den Hardware-Obergrenzen an. Die Ergebnisse werden in einer Dispatch-Policy zusammengeführt, die pro Workload-Regime die beste Konfiguration auswählt und auf 23 von 24 gemessenen Konfigurationen innerhalb von 5% validiert ist. Diese Engineering-Ergebnisse sind den strukturellen Ergebnissen nachgeordnet.

Der methodische Beitrag ist die *Pre-Flight-Obergrenzenidentifikation*: bevor eine Optimierung implementiert wird, sondieren wir die Zielkomposition mit Hardware-Performance-Countern und verwerfen jede Optimierung, deren Ziel-Obergrenze dort nicht mehr bindet. Diese Methodik ist nicht Fainder-spezifisch; sie gilt für hardwarenahe Systeme, deren Leistung durch mehrere gestapelte Obergrenzen begrenzt ist.

Abstract

Distribution-aware dataset search addresses the challenge of discovering datasets by statistical property. In a setting where the raw data is not accessible to the searcher (e.g., due to privacy or storage constraints), only per-column histograms are available. Fainder [9] introduces an index for this problem: it clusters histograms with K-means, aligns them onto a shared bin grid inside each, and pre-computes the cumulative-density arrays so that percentile predicates are answered by binary search, rather than histogram scans. Using the GitTables corpus of real-world histograms, it achieves a speedup of more than two orders of magnitude over a naïve profile-scan baseline, shrinking the index from over 1 TB to 3.2 GB. Fainder was originally written in Python, and the Python overhead is the dominant factor in its query time: the GIL (Global Interpreter Lock) serialises the per-cluster loop across threads, its AoS (Array-of-Structs) layout pollutes cache lines with unread hist-id fields, and per-iteration interpreter dispatch compounds onto both. This results in a 2.5-hour runtime for 10,000 queries on the full five-million-histogram workload, regardless of thread count.

This thesis investigates how Fainder’s query phase can be implemented efficiently on modern multi-core hardware, using a re-implementation in Rust. We find that query performance is bounded by four distinct hardware ceilings: single-thread load latency, shared cache capacity, memory-traffic pressure within a processor socket, and communication between sockets. Each ceiling binds at a different thread count and is addressed by a different class of interventions. We further find that optimisations addressing these ceilings compose *negatively*. Across five optimisation pairs, the same pattern repeats: a coarser optimisation has already absorbed the headroom on the target ceiling, and the finer optimisation’s overhead turns into a net regression rather than a diminishing return.

The re-engineered engine answers queries 530–598× faster end-to-end than the Python baseline while preserving exact results. Most of this ratio stems from avoiding Python’s per-result materialisation cost rather than from faster search. The search phase itself is up to 22.7× faster on small workloads and trends toward the hardware ceilings as the workload grows. The findings are integrated into a dispatch policy that selects the best configuration per workload regime, validated within 5% on 23 of 24 measured configurations. These engineering results are secondary to the structural findings above.

The methodological contribution is *pre-flight ceiling identification*: before imple-



menting an optimisation, we probe the target composition with hardware performance counters and discard any optimisation whose target ceiling no longer binds there. This methodology is not specific to Fainder. It applies to hardware-near systems whose performance is bound by multiple stacked ceilings.

Acknowledgments

First and foremost, I thank my advisor, Lennart Behme, for his guidance throughout this thesis, for his qualitative and honest feedback at every stage, for pushing me to make my claims as defensible as possible, and for the freedom to explore avenues that interested me. This work is much better for it.

Thank you to DIMA and its members for providing the resources and environment to make this work possible.

I thank my parents for their unconditional support throughout my studies, and for listening to my ramblings about performance engineering. None of this would exist without them.

Contents

List of Figures	xiv
List of Tables	xv
List of Abbreviations	xvi
1 Introduction	1
1.1 Motivation	2
1.2 Research Challenge	2
1.3 Scope	2
1.4 Four Performance Ceilings on Sapphire Rapids	3
1.5 Negative Composability and Dispatch	4
1.6 Contributions	4
1.7 Thesis Structure	5
2 Scientific Background	7
2.1 Distribution-Aware Dataset Search	7
2.1.1 Data Discovery and Distribution-Aware Search	7
2.1.2 Fainer Index	7
2.2 Hardware-Conscious System Design	8
2.2.1 Modern Superscalar CPU Architecture	8
2.2.2 Memory Hierarchy and Cache Efficiency	9
2.2.3 NUMA and the Cross-Socket Interconnect	11
2.2.4 Structure of Arrays vs. Array of Structs	12
2.2.5 Eytzinger BFS Layout	12
2.2.6 Branch Prediction and Branchless Code	14
2.2.7 Single-Instruction Multiple-Data (SIMD)	14
2.2.8 Hardware Performance Counters	15
2.2.9 Multicore Parallelism and Work-Stealing	16
2.2.10 Rust and PyO3	16
2.2.11 Binary Search Optimisation: <code>partition_point</code>	17
2.2.12 Rust Compiler Optimisations	17
2.3 Composability of Hardware-Conscious Optimisations	18



3	Methodology	19
3.1	Experimental Setup	19
3.1.1	Reproducibility	19
3.1.2	Hardware	19
3.1.3	Software	20
3.1.4	Datasets and Queries	21
3.2	Rust Engine Implementation	22
3.2.1	Engine Architecture	22
3.2.2	Cargo Feature Flags	22
3.2.3	Layout and Search Variants	23
3.2.4	Column-Centric Engine	25
3.2.5	Query-Batch Engine	25
3.2.6	Morsel-Driven Scheduler	26
3.2.7	Packed-IDs and Per-Cluster Reindexing	27
3.2.8	Memory Pooling and Allocation	28
3.2.9	NUMA Placement	29
3.2.10	PyO3 Integration	30
3.2.11	Flat-Binary Index Format	30
3.2.12	Rust Rebinning Kernel	31
3.3	Experimental Protocol	32
3.3.1	Perf-Counter Collection	32
3.3.2	Correctness Validation	32
3.3.3	Thread Sweep and Repetitions	32
3.3.4	Warm-Cache Regime	33
3.3.5	Suppress-Results vs With-Results	33
3.3.6	Confidence Intervals	33
3.4	Hypotheses	34
3.4.1	Ceiling (i) : Compute / L1 Latency	34
3.4.2	Ceiling (ii) : Shared L3 Capacity	34
3.4.3	Ceiling (iii) : On-Socket Memory-Traffic Pressure	35
3.4.4	Ceiling (iv) : Cross-Socket UPI	35
3.4.5	Negative-Composability Instances	35
3.4.6	Dispatcher and Out-of-Distribution Validation	36
4	Experimental Evaluation	37
4.1	Python Baseline	37
4.1.1	Wall-Clock at Three Scales	38
4.2	Reading Guide: The Four Ceilings	38
4.3	Ceiling (i): Single-Thread Compute / L1 Latency	40
4.3.1	Ablation A: AVX-512 Leaf-Stage SIMD	40
4.3.2	Ablation B: 8-Way Batch-Search	41



4.3.3	Ablation C: Horizontal-SIMD Lockstep	41
4.3.4	Ablation D: PGM Learned Index	42
4.3.5	Mechanism Evidence	42
4.4	Ceiling (ii): Shared L3 Capacity	43
4.4.1	SoA vs AoS	43
4.4.2	f16 Footprint Halving	44
4.4.3	Pooled Output Buffer	46
4.4.4	Mechanism Reading	47
4.5	Ceiling (iii): On-Socket Memory-Traffic Pressure	48
4.5.1	packed-ids : Bit-Packed Emit Phase	49
4.6	Ceiling (iv): Cross-Socket UPI Interconnect	50
4.6.1	Four-Config NUMA Probe and Results	50
4.7	Multi-Core Scaling and SMT Contention	51
4.7.1	HT-Sibling Contention and pin-cores	51
4.7.2	Nested Cluster Parallelism	52
4.8	The Work-Fragmentation Regime	52
4.8.1	Column-Centric Engine	53
4.8.2	Query-Batch: Cold-Load Amortisation	53
4.8.3	Morsel Scheduling	54
4.9	Negative Composability: A Structural Pattern	54
4.9.1	Five Instances on Five Axes	55
4.9.2	Arithmetic Pitfalls	55
4.9.3	Pre-Flight Ceiling Identification	56
4.9.4	Methodology and Recipe	56
4.10	Per-Cluster Reindexing (Falsified Prediction)	57
4.10.1	Pre-Registered Prediction	57
4.10.2	Four-Variant Measurement and Results	57
4.10.3	Mechanism: dTLB-Walk Stalls	58
4.11	Dispatch Policy: Derivation and In-Distribution Validation	58
4.11.1	Derivation	59
4.11.2	In-Distribution Validation	59
4.12	Out-of-Distribution Validation at c1024_56gb	60
4.12.1	Dispatch at the OOD Point	60
4.12.2	Structural Validation of Ceilings and Instances	60
4.13	best-combo vs Python Baseline	62
4.13.1	Python's Result-Materialisation Cost	63
4.14	Rust Rebinning Kernel	64
4.15	Variance Check on Borderline Claims	65
4.16	Summary	65



5	Related Work	68
5.1	Hardware-Optimised Index Structures	68
5.2	Vectorised and Parallel Query Processing	69
5.3	Approximate Histogram-Based Systems	70
5.4	Hardware-Aware Indexing of Adjacent Primitives	70
5.5	Composability of Optimisations in Database and Systems Engineering	71
5.6	Performance Gap in Distribution-Aware Search	72
6	Conclusion	74
6.1	Summary	74
6.2	Contributions	75
6.3	Limitations	76
6.4	Future Work	76
6.5	Closing Remark	77
	Bibliography	78
	Appendix A. Reproduction and Implementation Details	85

List of Figures

1	Memory-hierarchy access latencies on the measurement platform. . .	10
2	AoS vs. SoA memory layout.	12
3	Sorted vs. Eytzinger array layout.	13
4	Per-worker Chase-Lev dequeues in the morsel scheduler.	27
5	Search-phase wall time and the four ceiling regimes.	39
6	AoS wall-time delta across threads.	44
7	f16 fine- t wall-time delta.	46
8	End-to-end and search-phase speedup over Python.	64

List of Tables

1	The four evaluation datasets.	21
2	Cargo feature flags in the ablation matrix.	23
3	Index-loading formats and load times.	31
4	Chapter roadmap.	37
5	Python baseline wall-clock.	38
6	<code>simd</code> vs. default.	40
7	<code>batch-search</code> vs. columnar default.	41
8	<code>horizontal-simd</code> vs. columnar default.	41
9	<code>pgm</code> vs. default.	42
10	<code>aos</code> vs. default.	43
11	<code>pooled</code> vs. default.	46
12	<code>best-combo-minus-pooled</code> isolation.	47
13	<code>packed-ids</code> vs. default.	49
14	NUMA placement probe.	50
15	Thread scaling of the default build.	51
16	<code>cluster-par</code> vs. default.	52
17	Query-batch K -sensitivity.	53
18	Morsel vs. regime-best builds.	54
19	The five negative-composability instances.	55
20	Pre-registered bandwidth-budget arithmetic.	57
21	Per-cluster reindexing results.	58
22	Dispatch validation at the OOD point.	60
23	OOD structural-validation verdicts.	61
24	Speedup vs. the Python baseline at $t=16$	62
25	Rebinning kernel wall times.	64
26	Hypotheses and their verdicts.	66

List of Abbreviations

AoS	Array of Structs
AVX	Advanced Vector Extensions
BFS	Breadth-First Search
CI	Confidence Interval
CMOV	Conditional Move (instruction)
DIMA	Database Systems and Information Management
DRAM	Dynamic Random-Access Memory
dTLB	Data Translation Lookaside Buffer
FFI	Foreign Function Interface
GC	Garbage Collector
GIL	Global Interpreter Lock
HT	Hyper-Threading
IPC	Instructions Per Cycle
LLC	Last-Level Cache
MSHR	Miss Status Holding Register
NUMA	Non-Uniform Memory Access
OOD	Out-of-Distribution
PGM	Piecewise Geometric Model (learned index)
PMU	Performance Monitoring Unit
ROB	Reorder Buffer



SIMD	Single Instruction, Multiple Data
SMT	Simultaneous Multithreading
SoA	Structure of Arrays
TLB	Translation Lookaside Buffer
UPI	Ultra-Path Interconnect

1 Introduction

Open data repositories, scientific data lakes, and enterprise data catalogs have made very large collections of datasets available for search. In some of these settings, raw data itself cannot be searched directly: access may be restricted for reasons such as licensing or privacy. Even where access is possible, scanning millions of raw columns is simply impractical at interactive speed. In such settings, data providers publish statistical metadata instead (typically per-column histograms), and a searcher has to operate on these summaries. Distribution-aware dataset search [9] is the class of search problems whose predicates concern the distribution of column values. Percentile predicates of the form “*find all columns where the p -th percentile exceeds threshold t* ” are one such predicate class, and the one this thesis focuses on.

Fainder [9] introduces an index structure for this exact task. A collection of histograms is grouped into clusters using K-means. Each histogram in a cluster is then aligned onto a shared bin grid, and cumulative density arrays are precomputed so that percentile predicates can be answered via binary search, instead of needing to traverse histograms fully. On the GitTables benchmark of five million histograms, Fainder achieves more than two orders of magnitude speedup over a naïve profile-scan baseline while reducing memory footprint from over 1 TB to 3.2 GB.

This thesis is about accelerating Fainder. The goal is to investigate how Fainder’s query phase can be implemented efficiently on modern multi-core hardware. For this investigation, we re-implement Fainder’s query phase in Rust. The core of this thesis is the investigation itself: uncovering four binding performance ceilings on a current two-socket server, representative of the hardware class that search services deploy on (§3.1.2), and unearthing a pattern of *negative composability* across five optimisation axes. We develop a pre-flight ceiling-identification methodology from that pattern, and consolidate its findings into a deployment artefact that selects the best configuration per workload regime. Fainder is the concrete system under test, but we expect the methodology to transfer to any hardware-near system whose performance is governed by multiple stacked ceilings, while the specific thresholds we measure do not.



1.1 Motivation

Fainder’s original implementation [9] was implemented in Python, and uses NumPy. Using Python means that Fainder’s code is subject to the Global Interpreter Lock (GIL) [50], which is responsible for serialising bytecode across threads in a single process. Due to the GIL, multi-threading the per-cluster query loop cannot reduce wall-clock query time. The reference implementation also uses a memory layout that is not suited to Fainder’s access pattern, and per-iteration interpreter dispatch over the entire cluster chain compounds onto both.

All of this results in the query time of the reference implementation running in the interpreter, not in the binary-search algorithm it revolves around. Python-based Fainder takes hours (~ 2.5 hours for 10,000 queries) on the five-million-histogram primary workload on a modern two-socket server (§3.1.2), the GIL flattens the thread sweep so no thread count meaningfully reduces this wall. At a throughput floor of hours per workload, the interactive-exploration use case becomes untenable.

1.2 Research Challenge

The challenge in this thesis is to close the gap between Fainder’s algorithmic potential and the performance its reference implementation actually achieves, without sacrificing the correctness guarantees its existing use depends upon.

This research requires systematic low-level profiling to identify the dominant performance bottlenecks, and a replacement query engine that can make use of cache-conscious memory layouts and the parallelism provided by the hardware. We also need to validate that each of these implementations produces *bitwise-identical* results to the original reference implementation across all workloads.

A practical constraint, rather than a goal in itself, is integration: the rest of Fainder’s pipeline (clustering, data loading) remains in Python, so the query engine must stay callable from the existing Python API. The rebinning phase of index construction has additionally been ported to Rust (Chapter 3). K-means and histogram construction remain in Python.

1.3 Scope

In scope. The thesis focuses on re-engineering Fainder’s query-execution phase at a low system level and empirically evaluating hardware-oriented optimisations. We first profile the baseline implementation to identify the CPU and memory bottlenecks during percentile evaluation and binary search. Based on the profiling results, we introduce targeted optimisations, including cache-local data layouts for

the precomputed index, SIMD vectorisation of cumulative-density comparisons, and multicore parallelisation of independent cluster queries, and assess the performance impact of each, while maintaining exact correctness of query results. Histogram alignment belongs to the index-construction phase and is outside the primary optimisation focus. The alignment (rebinning) kernel was nevertheless ported to Rust, and its construction-time impact is evaluated separately (§4.14).

Out of scope. The thesis does not modify Fainder’s algorithmic foundations, mathematical model, or percentile-predicate semantics. It does not explore GPU acceleration, distributed computing, or cloud-based scaling approaches. User-interface design, visualisation, and non-performance-related extensions are likewise excluded. The work is confined to single-node, CPU-level optimisation, all improvements remain transparent to Fainder’s logical behaviour, so query results and accuracy remain identical to the reference implementation.

1.4 Four Performance Ceilings on Sapphire Rapids

The main empirical finding of this thesis (as unearthed by our ablations across more than twenty optimisation axes) is that distribution-aware search on a Sapphire Rapids server (§3.1.2) is subject to binding by **four performance ceilings**, each having a distinct binding under a different regime, addressed with a class of interventions.

- (i) **Single-thread compute / L1 latency.** At low thread counts ($t \leq 8$), the wall-time cost is the serial probe chain of binary search: each probe depends on the outcome of the previous comparison, so loads cannot overlap and the core spends its time waiting on cache accesses. This ceiling is the literature’s default assumption for binary-search workloads, and we include it as a candidate: on our workload and hardware, no latency-side intervention moves the wall beyond noise, so the candidate is refuted, which in itself a finding (§4.3).
- (ii) **Shared L3 capacity.** At moderate thread counts ($t \approx 16$) the per-thread working sets begin to overlap in the shared L3. Optimisations that reduce each thread’s memory footprint help against this ceiling, while optimisations that expand it are at a loss in this regime. Chapter 4 refines this ceiling into two regime-dependent mechanisms: allocator contention on dense clusters and aggregate L3 footprint at sparser densities (§4.4).
- (iii) **On-socket memory-traffic pressure.** At high thread counts ($t \in [32, 48]$) wall time rises as cores are added, characterised as aggregate memory-hierarchy pressure. Optimisations that reduce the bytes moved per comparison or per emitted result address this ceiling (§4.5).

- (iv) **Cross-socket UPI interconnect.** At thread counts that require both sockets ($t > 48$), the interconnect between the two NUMA nodes becomes a fourth ceiling. Single-socket placement removes part of the wall time, while naively interleaving memory across both sockets regresses (§4.6).

These four ceilings organise Chapter 4: we selected each optimisation axis to target one specific ceiling, so the ablation chapter reads as evidence for or against each ceiling hypothesis.

1.5 Negative Composability and Dispatch

This thesis’s principal *empirical* finding is the characterisation of four distinct performance ceilings on a modern two-socket server. The principal *structural* finding is that these ceilings are not independent: their interaction makes the composition of optimisations across ceilings non-trivial. Five ablation pairs, each on a different hardware axis, exhibit the same behaviour: a coarser optimisation has already addressed the binding ceiling, and the finer optimisation’s overhead ends with a net regression rather than diminishing returns. In each case, there was an argument for positive composition, which was then falsified by the **perf** counters. The methodological consequence of this pattern is *pre-flight ceiling identification*: probe the binding ceiling at the *target composition* before committing to implement a finer optimisation.

The dispatch policy aggregates these experimental findings into a deployment artefact: it selects a “best in regime” combination of build features and deployment flags for a given workload.

1.6 Contributions

The thesis’s contributions are organised as follows:

- (1) **Rust re-implementation of Fainder’s query pipeline.** A Rust implementation of Fainder’s query engine, preserving the index structure and interface compatibility with the Python API. We validated the engine to produce bitwise-identical results to the Python reference implementation across all workloads. At the peak-scaling regime, the end-to-end speedup over the Python baseline spans 530–598 $\times$. Most of this ratio is Rust avoiding Python’s per-result materialisation cost, and the search phase itself trends towards the shared hardware ceilings as per-cluster footprint grows. The Rust rebinning kernel achieves a separate speedup on the index-construction alignment phase. The engineering is presented in Chapter 3 as our main driver for the ablations.

- (2) **Four-ceiling micro-architectural characterisation.** Identification of four distinct performance ceilings via `perf` counters on a Sapphire Rapids (Xeon 8468H) server. Each of these ceilings binds under a regime that is addressable through a specific intervention. The evidence for the ceilings is presented in Chapter 4.
- (3) **Negative-composability pattern.** Five independent instances on five different axes of the same compositional shape, leading us to the result of negative composability of optimisation. This shows as a structural property of the access pattern rather than a property of any single ablation.
- (4) **Pre-flight ceiling-identification methodology.** We identify binding ceilings at the *target composition* using data from `perf`. These help us to avoid implementing an optimisation that is expected to be positive, but is actually negative at the composition baseline.
- (5) **Dispatch-on-regime deployment recipe.** An automatic dispatch that aggregates the experimental findings into a selection of build features and deployment flags, validated 17/18 exactly and 18/18 within 5% on the calibration data scales, and 6/6 at an out-of-distribution density point. Where contribution (4) is the transferable methodology, this recipe is its codebase-specific product: the regime boundaries and build selections hold for this workload and hardware, while the probing procedure behind them depends on neither.

The engineering artefacts (1 and 5) are the empirical staging ground for the scientific and methodological findings (2, 3, and 4). We would not have been able to identify the four ceilings without the Rust engine, nor identify the negative composability pattern without the ablations. The pattern itself is what generalises beyond this codebase.

1.7 Thesis Structure

The remainder of this thesis is organised as follows:

- 1. Chapter 2 provides the scientific background on distribution-aware dataset search, the Fainder index structure, and the hardware-conscious optimisation techniques applied in this work.
- 2. Chapter 3 documents the experimental setup (hardware, software, datasets, queries), the Rust implementation architecture, the measurement protocol, and the a-priori hypotheses each experiment tests.

3. Chapter 4 reports the experimental evaluation as evidence for each of the four ceilings identified in Section 1.4, with the negative-composability pattern in its closing section.
4. Chapter 5 surveys related work, with particular attention to the composability of optimisations in classical query optimisation and to the joint design of algorithms and hardware.
5. Chapter 6 concludes and outlines directions for future work, including the experiments most likely to confirm or extend the structural finding.
6. The appendix provides the reproduction and implementation details.

2 Scientific Background

This chapter reviews the material Chapter 4 builds on: distribution-aware dataset search and the Fainder index (§2.1), the hardware and systems-programming material relevant to the four ceilings (§2.2), and a primer on what it means for two hardware-conscious optimisations to compose (§2.3). A reader comfortable with these topics can jump to Chapter 3.

2.1 Distribution-Aware Dataset Search

2.1.1 Data Discovery and Distribution-Aware Search

Open data lakes and data collected in corporate catalogs are usually in the range of having millions of columns. Data scientists with a particular statistical profile of data in mind (for example, columns with a median between 50 and 100, or data in which the 90th percentile has a certain cutoff) would usually need to scan every file. *Distribution-aware dataset search* [9] is the class of search problems whose predicates concern the distribution of column values, rather than schema or keywords.

How such predicates are answered depends on what is accessible. For federated and privacy-sensitive data, the raw values are often not readable at all; the only available evidence is statistical metadata such as per-column histograms, computed before access is restricted [9]. This thesis targets that setting, and within it the percentile-predicate instance of distribution-aware search: given a collection of histograms $\mathcal{H} = \{h_1, \dots, h_n\}$ summarising individual columns, a query $\langle p, \bowtie, \tau \rangle$ must return every histogram h for which the p -th percentile of h satisfies $h[p] \bowtie \tau$, without ever touching the underlying rows.

2.1.2 Fainder Index

Fainder [9] is a three-phase index that exists for this type of query.

Index construction. In this phase, we group n histograms into k clusters by using K-means on the histogram vectors. In each cluster, we align the histograms onto a shared bin grid. Fainder makes use of two alignment strategies: *rebinning*, which projects each histogram onto a fixed bin count, which is compact and fast,

with a small alignment error near bin boundaries and *conversion*, which produces two index variants per cluster (a rebinned variant plus a higher-accuracy converted variant), at $\approx 2\times$ the memory cost and a 10–15% slower query. In this thesis we use rebinning throughout the entire work, since the accuracy delta stays the same throughout all of our findings.

Index structure. *FainderIndex* is a Union of all *SubIndex*’s. Each cluster c has a *SubIndex* that stores the cumulative density values of all histograms in c at each bin boundary b . The cumulative density values are stored in a sorted array, along with the matching histogram identifiers.

Query execution. To execute a query $\langle p, \bowtie, \tau \rangle$, the engine traverses through each cluster, where it searches the shared bin grid via binary search in order to find the bin b^* that corresponds to the reference value τ . Then, it performs another (binary) search on the cumulative density array at boundary b^* to collect the saved histogram IDs that follow the percentile predicate. The $\cup$ across all clusters is the final answer.

The query phase can be thought of as a sequence of binary searches, one per cluster. Because each cluster’s search is a chain of dependent loads, this thesis focuses on cache- and memory-hierarchy effects rather than SIMD throughput on streaming scans.

2.2 Hardware-Conscious System Design

In this section we cover the hardware and systems-programming background that Chapter 4 builds on. The goal of this section is to be a reference point for each of the ceilings introduced in Chapter 1. This also references the systems programming techniques used in the Rust implementation of *Fainder*, which is described in Chapter 3.

2.2.1 Modern Superscalar CPU Architecture

Modern server CPUs are out-of-order superscalar processors. The ancestor of these processors is Tomasulo’s 1967 dynamic-scheduling algorithm [57]. In the CPU, each core fetches and decodes instructions in the front-end and dispatches them to a reorder buffer (ROB). ROB depth scales with generation, from ~ 200 on Skylake to 512 entries on Golden Cove (the microarchitecture of the Sapphire Rapids Xeon 8468H used in this thesis). Each core has several execution ports that can perform multiple operations per cycle [27, 22]. Retirement width is also generation-dependent (4-wide on Skylake, up to 8-wide on Golden Cove). The IPC ceiling on this platform is therefore not the four-wide figure typical of older cores [29].



Instructions per cycle (IPC) is $\frac{\text{retired instructions}}{\text{cycles}}$. If a program is scalar and serial, and never stalls, it can approach the retirement width of the core (four on Skylake-class, higher on Golden Cove). The three main stall categories that push IPC below the ceiling in practice are:

1. *Front-end stalls*: the instruction fetch mechanism can't keep up, which usually happens after a branch misprediction (Section 2.2.6).
2. *Back-end stalls*: the execution ports are fully utilised, either because there are not enough independent instructions to fill them or because a dependency chain serialises the work.
3. *Memory stalls*: a load waits for data. L1 hit takes $\sim 4\text{--}5$ cycles, L2 hit $\sim 12\text{--}15$, L3 hit roughly 40. DRAM depends on locality, $\sim 170\text{--}320$ cycles for a local (on-socket) DDR5 access at 2.1–3.8 GHz, and roughly twice that for a remote access over UPI (see §2.2.3).

Work by Yasin [65] breaks the stall cycles into these categories. An IPC near the retirement ceiling indicates compute-bound work with healthy instruction-level parallelism, an IPC of ≈ 1 typically means the work is either heavily dependent or memory-bound.

If a cache miss stalls a load, the ROB can dispatch independent instructions to overlap the miss latency, modern CPUs rely on this to tolerate slow memory. The scenario where this cannot be used is when a load is *dependent* on the one before it, so load $i + 1$ cannot be issued until load i completes. Binary search is the canonical example: each step reads `arr[mid]`, and the next `mid` depends on the current comparison result, which serialises the load chain and defeats out-of-order execution.

2.2.2 Memory Hierarchy and Cache Efficiency

There are three layers of caches that sit between the core and DRAM, hierarchically:

- **L1 cache** is the smallest and fastest, typically tens of KB per core. It has the lowest latency, around 4–5 cycles for a hit.
- **L2 cache** is larger, typically a few MB per core. It has a higher latency, around 12–15 cycles for a hit.
- **L3 cache** is the largest and slowest, typically tens of MB shared across cores on the same socket. It has a higher latency, around 40 cycles for a hit.

Above all of these sits the DRAM [27, 16]. A cache line (which is essentially a block of memory that is transferred between the cache and DRAM) is typically 64 bytes wide. Manegold et al. looked at cache-conscious analysis for database operators [43].

Figure 1 demonstrates the access latencies of each level of the memory hierarchy on the platform used for this thesis. There are dotted vertical cutoffs that indicate the working-set size at which we involve the next, slower level of the hierarchy. The x axis is logarithmic, with the unit of nanoseconds, since we span six orders of magnitude from ~ 1 ns to ~ 5 ms. Due to the shift in order of magnitude, we can observe measurable wall-time changes when we shift a working set from L3 into DRAM, but any experiment that bottlenecks on disk-backed storage will be dominated by the disk latency, and the cache hierarchy will be largely irrelevant.

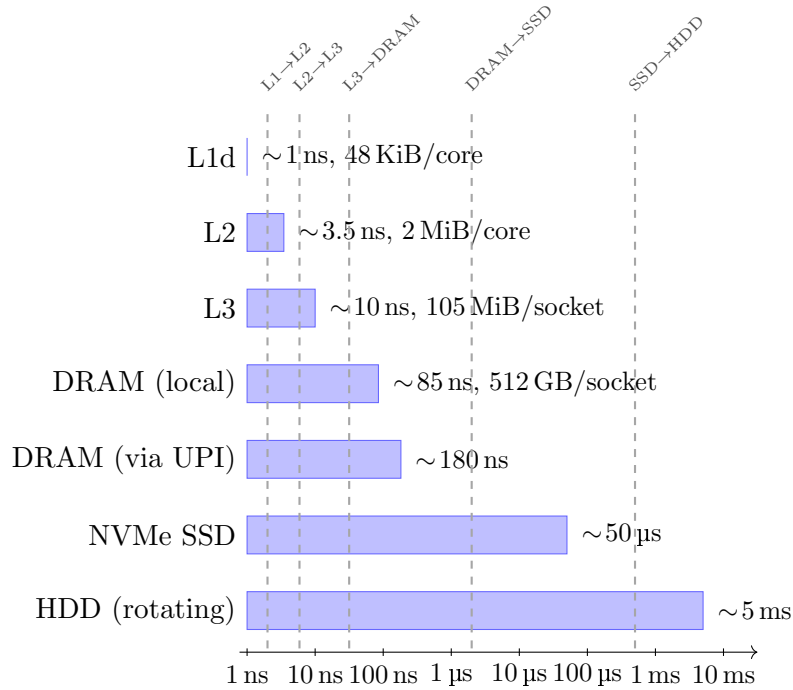


Figure 1: Access latencies along with the capacity of each level of the memory hierarchy on the Sapphire Rapids (Xeon 8468H) platform. X axis is logarithmic, with dotted lines along the Y axis indicating the working-set size at which the next slower level takes over (approximate transition-zones, not exact cutoffs). The latencies illustrated are approximate values for warm access, not peak throughput. Exact numbers may vary depending on prefetcher state, memory controller queueing, and NUMA policy.

Multiple prefetchers are attached to each core, they monitor access patterns and pull cache lines ahead of demand. L1 stride prefetchers catch sequential and constant-stride accesses (common in linear scans). L2 streamers catch coarser patterns. Neither can predict the data-dependent addresses of binary search, so binary search and other pointer-chasing or indirect-lookup workloads see little prefetcher help.

A core can hold multiple L1 misses in flight at once through Line-Fill Buffers (LFBs), their per-core count is ~ 10 on Skylake and ~ 16 on Golden Cove. Independent loads that all miss L1 perform round-trips to DRAM in parallel, which amortises the latency. A dependent load cannot benefit from this: the address of the next load is not known until the previous one is returned.

There is a fixed bytes-per-second budget for each DRAM channel. The Xeon 8468H used in this thesis has eight DDR5-4800 channels per socket, giving a per-socket peak of ~ 300 GB/s, and the two sockets together provide ~ 600 GB/s. When multiple cores stream data at once, that budget becomes contended and the effective per-core bandwidth drops. This differs from DRAM *latency*, which is the round-trip time of one load: bandwidth only becomes binding when the aggregate traffic across all cores approaches the channel capacity.

The main hot path in Fainder is a binary search over a sorted column of cumulative density values. With an SoA layout, each cache line loaded contains only percentile values. With an AoS layout, half of every line is histogram-IDs that the search phase never reads. The pollution halves the values-per-line ratio. Chapter 4 shows this layout effect is measurable but small once larger ceilings dominate at scale.

2.2.3 NUMA and the Cross-Socket Interconnect

Servers with multiple sockets split their DRAM into per-socket *NUMA nodes* [37]. Each socket reaches its local node at normal DRAM latency, but reaches the other socket's node with an additional round-trip over Intel's Ultra-Path Interconnect (UPI) link between the two CPU packages. On the platform used in this thesis, `numactl -H` reports a NUMA distance of 10 for local access and 21 for cross-socket access, roughly a $2.1\times$ latency penalty on remote loads. The NUMA performance pathologies that appear in multicore workloads when allocations and threads end up on different nodes are catalogued by McCurdy and Vetter [46].

This thesis uses a two-socket platform with 96 physical cores, and 48 cores per socket. The default Linux scheduler does not constrain a thread to a single socket, so any thread count above 48 will span both NUMA nodes out of necessity, which leads to cross-socket traffic. `numactl`'s `--membind` and `--cpunodebind` flags constrain a process to one node, which removes UPI traffic, but halves the core budget.

For our fourth ceiling in Chapter 4 we use the cross-socket interconnect as our source of contention. Both UPI and on-socket DRAM fall under the label “memory bandwidth”, but are binding under different conditions, and are measured differently. UPI has an empirical penalty that is distinguishable from the penalty for saturating local DRAM. The NUMA placement experiment of Chapter 4 separates the two.

2.2.4 Structure of Arrays vs. Array of Structs

SoA vs AoS is a classical concern when designing data-oriented systems [43]. Figure 2 illustrates the differences between them:

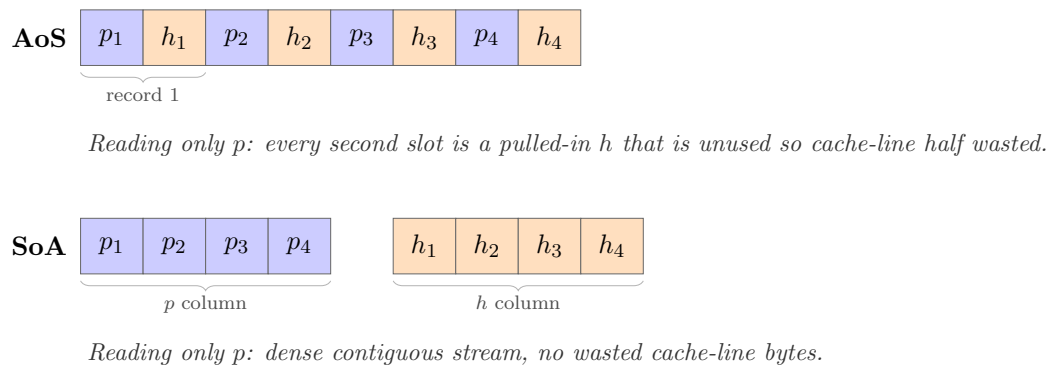


Figure 2: AoS interleaves fields per record and SoA stores each field in its own contiguous array. For Fainder’s access pattern (which touches only the percentile-value column (p) on every comparison), SoA halves the cache-line footprint by avoiding the unused hist-id (h) lanes.

SoA wins on cache utilisation when the access pattern is field-specific. For Fainder, only the percentile-value field is read during the search, and the histogram-ID is needed after the matching range has been located. SoA has about double the useful values per cache line. Chapter 4 quantifies this effect and shows it is regime-conditional: the advantage fades once memory-traffic pressure becomes the limiting ceiling at high thread counts.

2.2.5 Eytzinger BFS Layout

During binary search, sorted arrays are accessed in a pseudo-random pattern, which is bad for cache locality. There are unpredictable strides that make the hardware prefetcher ineffective, and each access hits a new cache line. The *Eytzinger BFS* (breadth-first) layout reorders a sorted array, such that the binary search tree is stored breadth-first. It stores the root at index 0, its children at 1 and 2, their

children at $3 \dots 6$, and so on [33]. We can store the first four levels of that tree in a single 64-byte cache line, that means that the first four bisections of an array cost one line load, instead of four. The L1 stride prefetcher can help with subsequent levels, since they access consecutive positions $8, 9, 10, \dots$ instead of scattered, random ones. Cache-oblivious analysis of layouts of this kind is attributed to research by Brodal and Fagerberg [13].

For searches of length n , this layout replaces $\log_2 n$ dependent-load stalls at pseudo-random offsets with a shorter sequence of loads at predictable stride, it also adds a bit-reversal permutation to convert the final BFS index back to the sorted-position index. Benchmarks by Khuong and Morin [33] show wins for Eytzinger at large n cold-cache searches, and small losses at small n where the binary search already fits in L1 after a miss. In Fainder’s per-cluster columns, Eytzinger’s cache-cold advantage largely disappears since the columns are L1-resident after the first cluster access in a query batch. An early campaign in this thesis included a Cargo-gated Eytzinger variant, but the feature was dropped from the final integrated build described in Chapter 3.

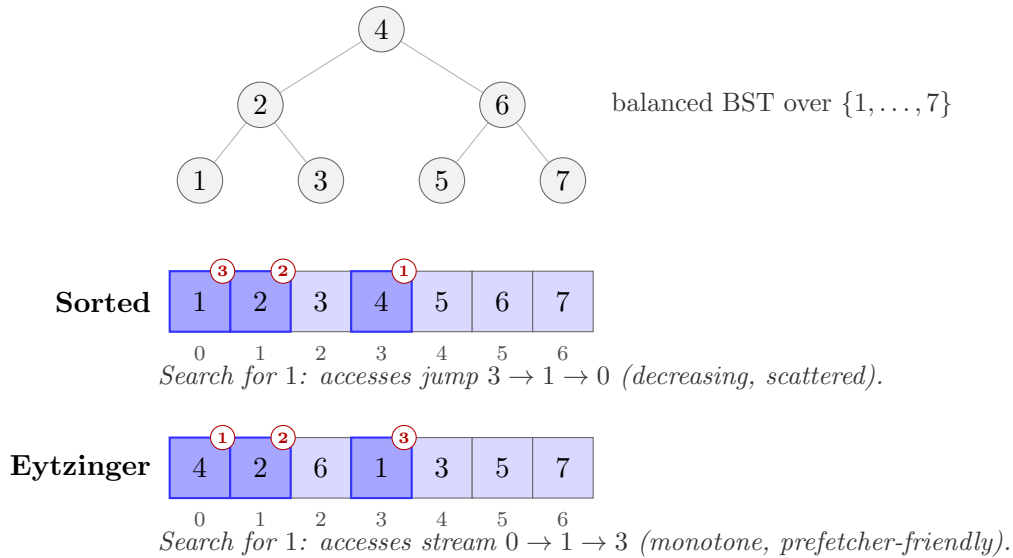


Figure 3: Sorted vs. Eytzinger (BFS) layout for a 7-element array. The Eytzinger layout stores the balanced BST in breadth-first order, so the root sits at index 0 and the early bisection nodes cluster in the low-index prefix. Binary-search accesses then form a prefetcher-friendly sequence instead of the pseudo-random jumps of the sorted layout.

2.2.6 Branch Prediction and Branchless Code

The front-end of a CPU pipeline fetches and decodes instructions. When a CPU encounters a conditional branch (for example an if statement), it has to decide which path it should follow before the conditional is evaluated, in order to prevent pipeline stalls. Modern CPUs employ branch prediction algorithms in order to predict the outcome of a branch, the branch predictor learns the history from previous executions [66] and predicts whether a branch will be taken or not. If the branch predictor guesses correctly, then the execution continues without interruption, however, if the prediction is incorrect, then the pipeline gets flushed by the CPU and the execution has to restart from a correct path, which is a significant performance penalty. This may cost 15–25 cycles, depending on the depth of the pipeline and the specific architecture.

CMOV. is the *Conditional move instruction* which is a branchless alternative to conditional branches. It selects between two register values based on a flag without emitting a branch, so misprediction is not possible. Standard binary-search routines in Rust (`slice::partition_point`), C++ (`std::lower_bound`), and NumPy’s C core (`numpy.searchsorted`) compile to a CMOV-based branchless loop. The following pseudocode illustrates this:

```
while size > 1 {
    half = size / 2;
    mid  = base + half;
    // CMOV: conditional update of base, no branch
    base = (arr[mid] < target) ? mid : base;
    size -= half;
}
```

This shows that the cost is not branching, but the chain of dependent L1 loads. Each iteration depends on the result of the previous comparison. For a per-cluster column of a few thousand elements the chain is $\log_2 n \approx 11\text{--}15$ sequential loads. This amounts to roughly 50–75 cycles per cluster search, and multiple thousand cycles per query. In Chapter 4 we confirm that the compiler emits the branchless loop, with a measured branch-misprediction rate of 0.01–0.03%.

2.2.7 Single-Instruction Multiple-Data (SIMD)

SIMD allows a single instruction to operate on multiple data elements in parallel [30]. Modern x86 CPUs expose SIMD extensions to accelerate vectorisable workloads. The most relevant SIMD extensions for this thesis are AVX2 and AVX-512. AVX2 employs 256-bit vector registers, and AVX-512 employs 512-bit vector registers. For example, an AVX2 `vcmpps` on `ymm` registers compares eight f32 lanes

against a broadcast scalar and yields an eight-bit mask via `movemask`, the equivalent AVX-512 `vcmps` on `zmm` compares sixteen `f32` lanes and writes directly to a sixteen-bit `k`-mask register. Zhou and Ross demonstrated systematic SIMD use in database operators [68]. Later work scaled SIMD to column scans [63] and to binary search [53].

Two SIMD patterns are relevant to binary search:

1. *SIMD within a search.* The last steps of binary search with fewer than eight elements remaining, can be vectorized. Those remaining elements are loaded into a vector register, compared against the target, and a bitmask is generated to locate the partition point. This reduces the number of iterations and dependent loads, improving performance when the data is cache-resident.
2. *SIMD across searches.* If we have multiple independent, parallel binary searches, the B -th steps of each search can be done in tandem. Since these loads do not depend on each other, MSHRs can pipeline them into parallel DRAM round-trips. Theoretically, this can hide up to $8\times$ of memory latency, but only if the bottleneck is actually memory latency.

Chapter 4 explores and evaluates both of these approaches, and concludes that for Fainder, the L1-resident columns are too small for SIMD to be effective. The first pattern is only relevant for the last few iterations of a search, which is a small fraction of the total search time. The second pattern requires multiple independent searches to be in-flight simultaneously, which is not the case for Fainder’s workload. The workload, instead, is bounded by the L1-latency dependency-chain length, which SIMD cannot shorten.

2.2.8 Hardware Performance Counters

Modern x86 CPUs have performance monitoring units (PMUs) for each core, which count low-level events such as retired (completed) instructions, cycles, branch-retired and branch-missed, cache-reference and cache-miss, and many more [29]. Linux’s `perf` tool [61] is able to sample these statistics and reports the aggregates. Yasin’s [65] method maps the raw counts onto stall categories, as mentioned in Section 2.2.1. Useful derived metrics include:

- $\text{IPC} = \frac{\text{retired instructions}}{\text{cycles}}$
- $\text{Branch-mispred rate} = \frac{\text{branch-misses}}{\text{branches}}$
- $\text{L1 miss rate} = \frac{\text{L1-dcache-load-misses}}{\text{L1-dcache-loads}}$



- **LLC miss rate** = $\frac{\text{LLC-load-misses}}{\text{LLC-loads}}$. An **LLC miss** is a load that misses the last-level cache (L3) and goes to DRAM. The LLC miss rate indicates whether the working set fits in the last-level cache. A high LLC miss rate indicates that the workload is memory-bound.

Counter multiplexing (the act of requesting more events than the PMU has hardware slots for) reduces accuracy. Chapter 4 explains how we adapted to work around it.

2.2.9 Multicore Parallelism and Work-Stealing

To exploit multicore parallelism on a CPU, a program needs either explicit threading or a work-stealing scheduler. Python’s GIL [50] serialises Python bytecode across all threads in a single process, that means that for CPU-bound Python code, “true” multicore parallelism is not achievable without leaving its interpreter.

Rayon [55] is a Rust library built on Blumofe and Leiserson’s work-stealing scheduler [11], and is designed for data-parallelism. Its `par_iter()` decomposes sequential computations into independent tasks and performs dynamic distribution across threads. Load imbalances are automatically handled by work-stealing. Since Rayon is Rust territory, the GIL does not apply.

We can map Fainder’s query execution onto Rayon easily, each query reads a shared, read-only index, with no cross-query data dependencies or shared mutable state. So locks, or atomic operations are not needed. Scheduling and load imbalances are handled by Rayon.

2.2.10 Rust and PyO3

Rust [44] is a systems language that gives performance on a level of C through (LLVM-based) native code generation and zero-cost abstractions. Memory and thread-safety are not a concern because of the ownership and borrowing type system. Rust does not possess a garbage collector, as memory is freed in a deterministic manner when its owning scope ends, so there are no GC pauses and tail latencies are predictable.

PyO3 [49] is a Rust crate that provides bidirectional Rust–Python FFI (Foreign Function Interface). To expose a Rust function to Python, it is annotated with `#[pyfunction]`, it is then compiled into a Python-callable extension, that makes it indistinguishable from a native Python function. The Maturin build tool [45] handles compilation and packaging. Other examples of this pattern include DuckDB [51], Polars [59], and Apache Arrow [4], which expose native cores to Python. PyO3’s zero-copy buffer uses NumPy’s array protocol [26] to share `numpy.ndarray` data with Rust.



2.2.11 Binary Search Optimisation: `partition_point`

Three-way binary search is a common algorithm for finding the partition point in a sorted array. Its first comparison is data-dependent on the sorted-array distribution, so there is usually a poor prediction rate. Rust's `slice::partition_point` takes a single boolean predicate `|x| x < target` and is then compiled to the CMOV-based branchless loop from Section 2.2.6. It does not emit a branch for the bisection update, and the only branch in this loop is the `while size > 1` terminator, which is highly predictable.

We can semantically equate this function to NumPy's `searchsorted(..., "left")` function, since it produces bitwise-identical results. We measure the misprediction rate of 0.01–0.03%, which confirms that the CMOV compilation reaches the release build.

2.2.12 Rust Compiler Optimisations

Cargo's `release` profile exposes several optimisation knobs. This thesis's build uses:

- `opt-level = 3` (Cargo default for `release`): enables aggressive optimisations, including inlining, loop unrolling, and auto-vectorisation.
- `-C target-cpu = native` (via `.cargo/config.toml rustflags`): optimises the generated code for the host CPU architecture, enabling AVX2, AVX-512, BMI2, FMA, and CPU-specific tuning on the Sapphire Rapids server.

Two further knobs Cargo exposes `lto = true` (link-time optimisation, whole-program analysis across crate boundaries) and `codegen-units = 1` (reduced code-generation parallelism for better inter-unit optimisation) are *not* enabled in this build. The measurements in Chapter 4 therefore reflect the default Cargo release profile.

The release profile also permits SIMD intrinsics and auto-vectorisation of compatible inner loops. *However*, Fainder's binary-search bottleneck is a chain of dependent loads that the compiler cannot vectorise across iterations (due to data dependencies), the explicit AVX-512 leaf-stage variant is evaluated separately in Chapter 4.

2.3 Composability of Hardware-Conscious Optimisations

Chapter 4 finds that five optimisation pairs, each with a clean a-priori argument for positive composition, do *not* compose positively. This is a structural finding, and it is the main empirical contribution of this thesis. The pattern of composability is consistent enough that the thesis names it *negative composability* and treats it as a property of hardware-near systems with multiple stacked ceilings. This section introduces what “composing” two optimisations means and why the classical textbook argument for positive composition can fail in the presence of multiple ceilings.

The textbook expectation is to view optimisations as independent actions that each save a measurable resource. For example:

- An SoA layout saves cache-line bytes.
- f16 representations halve memory-stage bandwidth.
- An ID-bit-packed array shrinks the emit-phase footprint.

In the case of the bandwidth-budget, having optimisation A , which reduces bandwidth use by a factor of r_A , and optimisation B , which reduces it by r_B , classically we would expect that applying both should leave $(1 - r_A)(1 - r_B)$ of the original demand. As long as the workload remains bandwidth-bound, the composition is at worst diminishing-returns positive: the gain from B on top of A is smaller than the gain from B alone, but it stays non-negative.

This argument assumes that the binding resource is the same before and after A is applied. In a system such as ours where there are multiple ceilings, the assumption can fail: the binding constraint can shift to a different ceiling after A is applied, and B may not address that new ceiling. In such cases, the effect of B can be negative, because B ’s non-zero costs on the new binding ceiling become visible once the slack that hid them before has been consumed by A .

Chapter 4 documents this failure empirically on five different hardware axes, and derives from it a pre-flight ceiling-identification methodology that probes the binding ceiling at the target composition before implementation cost is committed.

This framework is not specific to Fainder, and it is likely to be relevant for other hardware-near systems that have multiple ceilings. The thesis lays no claim to the universality of negative composability, but instead provides concrete examples of how it manifests in practical systems, and how to reason about it.

3 Methodology

This chapter documents the experimental methodology for the results produced in Chapter 4. Unlike Chapter 2, which describes the concepts and techniques used in the ablation study, this chapter gives the concrete hardware and software configuration, the Rust engine’s implementation architecture, and the measurement protocol for the reported numbers. In addition, we also describe all a-priori hypotheses that each experiment tests.

The main drive of this chapter is *reproducibility*: a reader with the same hardware, source data, and scripts should be able to reproduce every table and figure from Chapter 4 within the reported 95% confidence intervals.

3.1 Experimental Setup

3.1.1 Reproducibility

All measurement scripts, the Rust engine source code, and the raw measurement database are archived in the accompanying artefact, available at <https://github.com/tmukh/Fainder-Redone>. A companion set of walkthrough notebooks reproduces every table and figure in Chapter 4 directly from the measurement database.

3.1.2 Hardware

The hardware configuration is as follows

- **CPU:** 2× Intel Xeon Platinum 8468H (Sapphire Rapids-SP), 96 physical cores, 192 logical cores, 105 MiB L3 cache per socket, 48 KiB L1d and 2 MiB L2 caches per core, AVX-512 including AVX-512_FP16.
- **Memory:** 1 TB DDR5-4800, 8 channels per socket, ~ 300 GB/s per-socket peak bandwidth, ~ 600 GB/s aggregate.
- **Interconnect:** Ultra-Path Interconnect (UPI) between sockets, NUMA distance 10 for local access, 21 for cross-socket access ($\sim 2.1\times$ latency penalty on remote loads).



- **NUMA topology:** two nodes, one per socket. Node 0 holds logical CPUs 0–47 with SMT siblings 96–143, node 1 holds 48–95 with siblings 144–191.
- **Storage:** NVMe SSD (dataset loads) and a RAM-backed tmpfs at `/dev/shm`.
- **Operating System:** Ubuntu 22.04.5 LTS, Linux kernel 5.15.0-176-generic, glibc 2.35.

The Rayon thread pool for the thread-count sweeps in Chapter 4 is set explicitly via `FAINDER_NUM_THREADS`. Thread counts up to 48 fit within one NUMA node’s physical core budget, and 64 and above span both sockets, which requires UPI traffic. This clearly separates the NUMA experiment of Chapter 4 from the fourth ceiling.

3.1.3 Software

The software configuration is as follows:

- **Rust:** 1.94.1 (stable, LLVM 21.1.8), compiled with `rustc` in release mode with target-specific tuning for Sapphire Rapids.
- **Cargo:** 1.94.1, used to manage the Rust engine’s build and feature flags.
- **Maturin:** 1.7, used to compile and install the Rust engine as a PyO3 extension into the active virtualenv.
- **Python:** 3.10.12 with NumPy 1.26.4, used to run the reference Fainder implementation and orchestrate the benchmarking scripts.
- **Fainder:** The reference implementation of Fainder [9], used as the baseline for performance comparisons.
- **Benchmarking Scripts:** Custom Python scripts to automate the execution of benchmarks, collect performance data, and store results in a SQLite database.
- **Performance Counters:** `perf` tool to collect hardware performance counters during benchmark runs.
- **Data Compression:** `zstd` for compressing and decompressing query files.

3.1.4 Datasets and Queries

Both the original Fainder paper and this thesis make use of the *GitTables* [28] corpus, a public collection of $\sim 1\text{M}$ tables ($\sim 120\text{M}$ columns) extracted from public GitHub CSV repositories. After filtering to numeric columns with ≥ 100 rows, the subset used in this thesis contains 5,017,619 histograms.

From this corpus we derive four datasets, all through the same pipeline:

histogram extraction $\rightarrow$ MiniBatchKMeans clustering $\rightarrow$ per-cluster rebinning to a shared bin grid.

The derivation process is as follows:

1. Using K-means, histograms are clustered into $k = 256$ clusters, and all histograms are rebinned to a fixed bin count of 256, producing the **c256_56gb** dataset.
2. Subsequently, we use clustering again to cluster the histograms into $k = 1024$ clusters, and rebin each histogram into a bin count of 1024. This produces the **c1024_56gb** dataset.
3. Lastly, to create the **c256_30gb** and **c256_10gb** datasets, we sub-sample the corpus down to $\sim 30\text{ GB}$ and $\sim 10\text{ GB}$ of raw data (996,632 and 323,719 histograms), and re-do the K-means clustering and rebinning to $k = 256$ clusters and a fixed bin count of 256.

In the naming convention, the first number is the K-means target k , and the second number is the approximate size of the dataset in gigabytes. The variation between these datasets is two-fold, one is the total number of histograms, and the other is the *effective* number of clusters. Note that the target cluster count k is not the number of clusters actually populated in the index: some clusters may be empty, and small clusters may be merged. The populated count, reported as “effective clusters” in Table 1, is the one used at query time.

Table 1: The four calibration and OOD datasets. The primary workload is **c256_56gb** on which the findings are established. **c1024_56gb** is the out-of-distribution density point (§4.12). *Raw Parquet* is the source data size, *per-cluster f32* the working-set footprint of a single cluster’s column slice at f32 precision.

Dataset	Raw Parquet	n_{hists}	K_{target}	K_{eff}	hists/cluster	per-cluster f32	Role
c256_10gb	$\sim 10\text{ GB}$	323 719	256	129	$\sim 2.5\text{ K}$	$\sim 10\text{ KiB}$ (L1d)	small scale
c256_30gb	$\sim 30\text{ GB}$	996 632	256	184	$\sim 5.4\text{ K}$	$\sim 22\text{ KiB}$ (L1d)	medium
c256_56gb	$\sim 56\text{ GB}$	5 017 619	256	191	$\sim 26\text{ K}$	$\sim 105\text{ KiB}$ ($>\text{L1d}$)	primary
c1024_56gb	$\sim 56\text{ GB}$	5 017 619	1024	610	$\sim 8.2\text{ K}$	$\sim 33\text{ KiB}$ (L1d)	OOD (§4.12)



The query workload is a fixed set of 10,000 (*percentile*, *op*, *threshold*) tuples of the form (p, op, τ) with $p \in [0, 1]$, $op \in \{<, >\}$, and $\tau \in [\text{domain min}, \text{domain max}]$, sampled from the query workload used in the original Fainder paper and shared across all four datasets so the query characteristics are constant. Queries are dataset-independent: no column IDs appear in the query schema.

3.2 Rust Engine Implementation

In this section we describe the implementation of the concepts from Chapter 2 into Rust source code, sufficiently enough for readers to interpret the ablation labels in the Chapter 4. This section should not be considered an exhaustive overview of the codebase, that can be found in the accompanying artefact (§3.1). Instead, we focus on the architectural decisions that lead us to determine which measurements are meaningful and which cargo features they map to.

3.2.1 Engine Architecture

The index is loaded only once, through a memory-mapped flat binary file (§3.2.11). Query batches are answered through the Rayon work-stealing scheduler. A query batch is a Python-side `Vec<PercentileQuery>`, and its results are collected into a Python-side `Vec<Vec<u32>>` of matching histogram identifiers per query. By default, the row-centric engine spawns one Rayon task per *query* and processes clusters sequentially inside each task (`par_iter` over queries). Feature-gated variants swap this out for column-centric iteration (§3.2.4), K-query batch amortisation (§3.2.5), or morsel-driven dynamic dispatch (§3.2.6), each of which inverts the loop order so clusters become the outer dimension.

3.2.2 Cargo Feature Flags

Each ablation is mapped onto one or more Cargo feature flags. In table 2 we list the flags that appear in the `bench.db` labels. An absence of flags denotes the `default` build: SoA layout, scalar `partition_point`, glibc allocator and no thread pinning.

Table 2: Cargo feature flags exercised in the ablation matrix. Each flag targets a specific ceiling, the mapping is documented in §3.4.

Flag	Effect
(default)	SoA layout, scalar <code>partition_point</code> , glibc malloc
<code>simd</code>	AVX-512 leaf-stage compare in the binary-search inner loop
<code>f16</code>	Percentile values stored as f16 (half-precision), halves index size
<code>aos</code>	Array-of-Structs layout (negative control for L3 pressure)
<code>packed-ids</code>	Per-cluster bit-packed identifiers, $\lceil \log_2(\max_id + 1) \rceil$ bits
<code>pooled</code>	Preallocated per-query output buffer, no <code>flat_map+collect</code>
<code>pin-cores</code>	Rayon workers pinned to distinct physical cores (avoids SMT siblings)
<code>cluster-prefetch</code>	Software <code>_mm_prefetch</code> for cluster $c+1$ while c processes
<code>mimalloc</code>	mimalloc allocator with per-thread sharded free-lists
<code>query-batch</code>	Column-centric inner loop within K-query batches (K per env var, default 64)
<code>morsel</code>	Phase-aligned cooperative-caching scheduler (Chase-Lev)
<code>local-ids, -bench, -noalloc</code>	Per-cluster identifier reindexing (three mutually-exclusive variants §3.2.7)
<code>horizontal-simd</code>	AVX-512 over 16 queries in lockstep (Polychroniou-style)
<code>pgm</code>	Learned-index acceleration of binary search

The composed build `best-combo` is not a Cargo feature but a build-preset shorthand for the combination `--features "pooled f16 simd pin-cores cluster-prefetch mimalloc"` (the integrated optimum used throughout Chapter 4 named `bestofsuite` in the artefact and in the measurement-database labels).

3.2.3 Layout and Search Variants

The concepts of SoA, AoS, and SIMD are introduced in Chapter 2. Below we describe the implementation of each as a Cargo-feature-gated variant of the `SubIndex` struct and the leaf-stage binary-search function. The Eytzinger BFS layout and the 4-way branchless k-ary search were part of an earlier campaign, both were dropped from the final integrated build and are not part of the final measurement campaign.

The default build’s SoA’s `SubIndex` is column-major: so for a cluster with n histograms and b bin boundaries, the array of values hold $n \times b$ f32 values, so that all values for boundary k occupy the slice `values[k*n .. (k+1)*n]`, and the



array of identifiers is separate.

```
pub struct SubIndex {  
    pub values: Vec<f32>, // column-major  
    pub indices: Vec<u32>, // same layout as values  
}
```

The binary-search predicate reads only from the `values` slice and identifiers are touched once at emit time to collect the matching histogram IDs.

The following Cargo feature flags implement layout variants:

- `aos`, replaces `SubIndex` with an AoS layout: `Vec<(f32,u32)>` interleaved value-ID pairs.
- `f16`, replaces `values: Vec<f32>` with `values: Vec<half::f16>`, with on-the-fly dequantisation via `x.to_f32()`.

Since all these variants implement the same trait, the outer engine code is unchanged, the compile-time feature selection swaps the concrete `SubIndex` type.

Rust's `slice::partition_point` is the binary search itself. It's a branch-less variant that can be equated (semantically) to NumPy's `searchsorted` with `side="left"`. The binary search, like the index, has feature flags as well:

- `simd`, replaces the last ~ 16 iterations of `partition_point` with one AVX-512 vector comparison: `_mm512_cmp_ps_mask` (predicate `_CMP_LT_OQ`) for f32 and `_mm512_cmplt_epu16_mask` on reinterpreted f16 bits.
- `horizontal-simd`, implements the Polychroniou-style [48] multi-probe: 16 queries traverse their bisection trees in tandem through one column, sharing prefix loads.
- `batch-search`, interleaves eight serial searches on the same column so that later-stage independent loads pipeline through the CPU's Miss Status Holding Registers.
- `pgm`, instead of a load chain that is dependent on the previous comparison, the search now becomes a piece-wise linear segment lookup plus a local scan. The PGM segments are built once when the index is loaded, and stored alongside the column. If the hardware does not support AVX-512, the search falls back to the scalar variant.

If a query's reference value falls outside the cluster's bin-edge range, the search returns `bin_idx = 0` or `bin_idx = n_bins`, and the inner loop short-circuits to `cluster_all_ids(c)` or `vec![]` without any binary search. This edge case caused a precision/recall bug in early versions and is now covered by the correctness gate (§3.3.2).



3.2.4 Column-Centric Engine

In the default row-centric engine, queries are the outer parallel loop and clusters are the sequential inner loop within each Rayon task. This means that for each query, the engine iterates over all clusters, loading each cluster's bin edges and column slice into L1/L2 cache. When the next query is processed, the previously loaded cluster data may have been evicted from cache, leading to repeated cache misses and increased memory latency. The column-centric engine is selected by setting the environment variable `FAINDER_COLUMNAR=1`. The engine inverts the loop order, making clusters the outer parallel dimension and queries the inner loop, leading to better cache locality, as all queries for a given cluster can be processed while that cluster's data remains in cache.

The four inner steps are as follows:

1. Pre-compute k^* for each query on this cluster.
2. Group queries by k^* and sort within-group by operator.
3. For each bin-index group, run a binary search over `values[k*n .. (k+1)*n]` for all queries in the group sequentially so the column slice stays L2-warm.
4. Write per-query match counts and flat result arrays into a cluster-owned buffer with a separate offsets array.

If we use a flat buffer for the output, we avoid the row-centric default's N_{queries} small allocations per cluster, instead we allocate `Vec<u32>` per cluster plus an offsets vector. Since `batch-search` only makes sense when consecutive searches actually hit the same column slice, the `--features batch-search` SIMD variant is meaningful only in the column-centric engine.

At query-invocation time, the engine-selection check reads the `columnar` boolean (driven by `FAINDER_COLUMNAR`) and the compile-time presence of the `query-batch` / `morsel` features to select the appropriate engine, ensuring that exactly one engine handles any given `run_queries` call. (A separate routing component performs a different, finer-grained job: per-(query, cluster) route computation for early pruning and trivially-all short-circuits.) The engine-selection mechanism exists because the regime grid does not have a single dominating engine across all scenarios. The deployment recipe (§4.11) picks the engine at the time the workload is dispatched, not at compile time.

3.2.5 Query-Batch Engine

Due to the columnar engine's reuse of a cluster's column slice within one outer task, once a thread advances to the next cluster, that data is gone. With `--features`

query-batch we keep the row-centric strength of parallelism, but also group consecutive queries into batches of size K and amortise each cluster col-load across all K queries in the batch.

Over the query list, the outer parallel iterator is now **par_chunks**, instead of **par_iter()** over individual queries. We now invert the loop order inside each chunk to cluster-outer + queries-inner: worker visits cluster $c \rightarrow$ run all queries against c while bin-edges and column slice are cache-warm $\rightarrow$ advance to cluster $c+1$. The batch size parameter defaults to $K=64$ and can be overridden via **FAINDER_QUERY_BATCH**. The sweep range for the experiments is shown in §4.8.

Correctness is byte-identical to the default build, validated on 10K queries against the row-centric reference at $t=16$ (§3.3.2).

3.2.6 Morsel-Driven Scheduler

Unlike the query-batch engine, which amortises cluster loads within one Rayon task, the morsel-driven engine amortises cluster loads across all Rayon tasks. In the query-batch scenario, if two workers are processing the same cluster in parallel, each worker will load the cluster’s data into its own L1/L2 cache, leading to redundant memory accesses (L3 is shared, but L1/L2 are private).

We can address this by implementing a phase-aligned scheduler, which ensures that all workers process the same cluster before moving on to the next one, keeping the cluster’s data hot in the shared L3 cache. The morsel engine (**--features morsel**) implements this pattern, following the HyPer/Umbra morsel-driven design [39, 7].

A morsel is the pair (cluster c , queries $[q_a..q_b]$). The implementation consists of four main components:

- **Pre-bucketed phase-major queue.** A cluster is processed in the order of largest processing time first (Graham 1969 [24]), binding the time the last worker finishes by $4/3 \times \text{OPT}$. We chunk queries in each cluster into *morsels* by a sizing heuristic resulting in ~ 50 queries on the largest clusters, scaling as the inverse-log of **n_hists** on smaller ones.
- **Per-worker Chase-Lev deque** Before any worker is spawned, all morsels are pushed back round-robin across the per-worker deque, in the order of largest time to process. Each worker then pops its own morsels FIFO from the front. A worker whose deque drains round-robin steals from the front of a peer’s deque via **Stealer::steal**.
- **Soft phase alignment.** No explicit barrier: if all workers start at phase 0 and consume FIFO from their own deque, they advance through phases



roughly in tandem. Stragglers steal single elements across phases via `Stealer::steal`.

- **Per-worker append-only output buffers.** Each pair of (q, c) is processed by one worker, which writes the matching IDs into a per-worker flat buffer. No shared output buffer means no contention on emit

An implementation using per-worker $\times$ per-query `Vec<Vec<u32>>` produced hundreds of thousands of `Vec` headers per worker, each hitting a different cache line. Allocator pressure in this scenario was abysmal, leading to a 10–15 $\times$ slow-down compared to the default build.

Another (v2) attempt using flat-buffer plus chunk-index (`Vec<u32>` plus `Vec<usize>`) alleviates the cost of the allocator pressure, but at the cost of a more complex merge phase. We validate the correctness of this approach against the default build on `c256_10gb` at $t=16$, confirming byte-identical results.

main thread pre-loads morsels round-robin (LPT-sorted, phase-major) before spawning workers

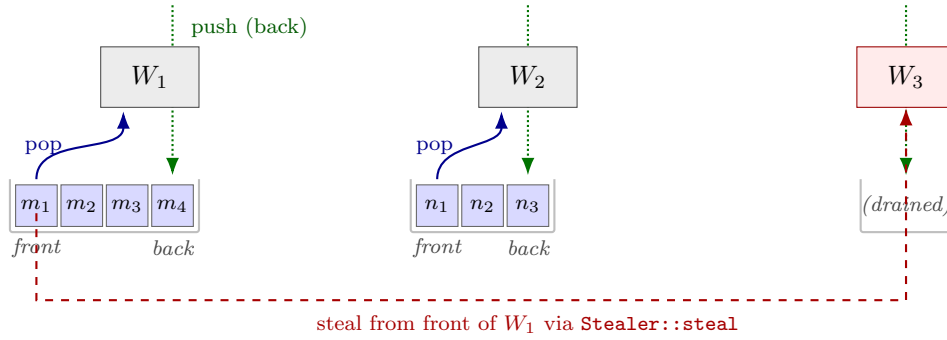


Figure 4: Per-worker Chase–Lev dequeues in the morsel scheduler.

3.2.7 Packed-IDs and Per-Cluster Reindexing

The variants introduced in §3.2.3 all target the *values* side of the index (how percentile columns are laid out and how they are searched). This section targets the other half, the *emit* phase that copies matching histogram IDs out of a cluster into the result buffer once a match is found.

Emit brings cost with it, since the default engine copies 32-bit IDs verbatim. On the primary workload, the write becomes a measurable share of the end-to-end time at high thread counts. Precision is also wasted since no cluster contains anywhere near 2^{32} histograms, so most of every emitted word is zero.



Bit-packing the ID column. When building with `packed-ids`, we shrink each ID to $\text{ceil}(\log_2(\text{max_id} + 1))$ bits (19–23 bits across the calibration datasets, and 23 bits on the primary workload `c256_56gb`). The IDs are column-major packed with a trailing padding word so that the decoder’s word-sized loads can never overrun the buffer. This turns the emit into a three instruction sequence for each ID: (1) load a 64-bit word, (2) shift, (3) mask.

Sizing the width correctly matters more than packing itself. If we size by cluster cardinality rather than by the largest ID present, high bits are silently truncated (due to the bit-width limitation): the engine compiles, completes, achieves $\sim \frac{1}{6}$ of the correct results. Width is now sized by the observed maximum ID, and the correctness gate catches any mis-sizing.

Reindexing to local IDs. If we bit-pack the global IDs, we get down to ~ 20 –23 bits (dataset-dependent). If each cluster reindexes its IDs to a local $[0, n_{\text{hist}})$ range, the width drops further to ~ 13 bits. We run into the issue that downstream consumers expect global IDs, so the emitter has to translate through a per-cluster lookup table. This introduces two new costs: the lookup instructions, and the memory footprint of holding every cluster’s translation table resident.

A four-way decomposition. We expose four build modes instead of committing to a single `local-ids` variant, so each cost can be measured in isolation. This lets us attribute any observed regression to a specific mechanism instead of just a black-box net effect. The four modes are the `packed-ids` baseline plus three `local-ids` variants:

- *Global 20-bit:* baseline (`packed-ids`). No reindexing, no lookup.
- *Local 13-bit, correct:* `local-ids`. Reindexed emit reads a 13-bit local ID and translates through the per-cluster table, byte-identical.
- *Local 13-bit, no lookup:* `local-ids-bench`. Emits the local ID verbatim, so the output is *incorrect* and this mode is only valid under `FAINDER_SUPPRESS_RESULTS=1`. Isolates the pure bandwidth win from the lookup cost.
- *Local 13-bit, table absent:* `local-ids-noalloc`. Same emit path as `-bench`, but the translation table is never allocated.

3.2.8 Memory Pooling and Allocation

Our implementation does not build a generic arena allocator, instead, it embeds memory-pooling principles in four places:



- **Columnar flat buffer.** The column-centric engine pre-allocates one `Vec<u32>` per cluster (with an offsets array) instead of one `Vec<u32>` per (query, cluster) pair. Instead of hundreds of thousands of small heap allocations per query batch, we have a handful of large pre-allocated buffers, eliminating glibc arena mutex contention.
- **Pooled output buffer.** We pre-allocate one `Vec<u32>` per query, and extend it from each cluster's matching range. This avoids the per-cluster temporary `Vec<u32>` chain that the default row-centric engine produces via `(0..n_clusters).flat_map(process_cluster).collect()`.
- **Flat-binary mmap.** Python deserialisation of the index is a bottleneck for the row-centric engine. This is addressed by the `.fidx` flat-binary format (§3.2.11), which `load_index()` detects at runtime by file extension and memory-maps in place of walking a Python pickle tree.
- **Rebinning kernel.** The Rust rebinning kernel (§3.2.12) allocates, for each cluster, a single Fortran-order flat buffer and writes all cumsum values directly into it. This replaces the per-histogram intermediate arrays that NumPy's `argsort(axis=0)` would otherwise create.

3.2.9 NUMA Placement

Our two-socket platform has two NUMA nodes, each with its own DDR5 memory controller. This means that we are in possession of two memory bandwidths: (1) on-socket DDR5 (~ 300 GB/s per socket) and (2) cross-socket UPI. As mentioned, Linux and Rayon cause a worker on socket 1 to read bytes allocated on socket 0 at index-load time. This cross-socket traffic looks like DRAM bandwidth pressure in coarse counters but actually saturates a separate, smaller pipe.

We can run four different `numactl` configurations to probe the UPI link's effect on throughput:

- **Unpinned.** Linux first-touch, default scheduler.
- **Single-socket-clean** (`numactl --cpunodebind=0 --membind=0`). Memory and CPUs both pinned to node 0, helps us identify if the UPI link is a bottleneck.
- **Interleave 0,1** (`numactl --interleave=0,1`). Performs "Balance memory across sockets". Pages are allocated round-robin across both sockets.
- **Cross-socket forced** (`numactl --membind=0 --physcpubind=48..N`). Memory on node 0, CPUs on node 1. Worst-case configuration for UPI cost. Isolates the link as the binding pipe.



The pre-flight ceiling-identification methodology (§4.9.3) systematises this four-configuration probe. It is able to tell whether the UPI link is a bottleneck, and if so, whether it is due to the link itself or due to the memory controller on the remote socket.

3.2.10 PyO3 Integration

In combination with Maturin, PyO3 lets us write the query engine in Rust and expose it to Python as a native extension module. The index constructor accepts the pre-built index as a Python list of NumPy arrays and copies values into Rust-native `Vec` storage. Type Conversions (f16 cast, bit packing) are performed during ingestion. After construction, the index is read-only and shared across threads.

The Python wrapper exposes a single `run_queries` function that accepts a list of queries and returns a list of result arrays. The engine is fully parallelised using Rayon, and the GIL is released during query execution. After execution, the GIL is re-acquired to return the results to Python as a Python list of `numpy.ndarray` arrays. The overhead is a fixed ~ 2 ms regardless of dataset size.

The reason we return `numpy.ndarray` instead of a Python `set` is that the latter would require a per-element `PyObject` allocation, which is expensive. Instead, it performs a single `memcpy` from the Rust `Vec<u32>` into a NumPy buffer.

3.2.11 Flat-Binary Index Format

The original Fainder index serialises as a single `pickle+zstd` file. To load, it decompresses ~ 500 MB of `zstd` stream into ~ 25 GB of Python object tree (nested lists of tuples of NumPy arrays). This requires Python to allocate wrapper objects for every list, tuple, and array in the hierarchy. For `c256_30gb` this takes about 15 s per load. For experiments that run the same index repeatedly (a seven-point thread sweep, for instance) this adds ~ 100 s of avoidable I/O overhead. The flat-binary format addresses this by storing the index as a directory of raw `.npy` files, which can be memory-mapped and loaded on demand.

This flat index is a collection of `.npy` files, one bin per cluster, two arrays per cluster per mode (percentiles and IDs), and a small `meta.json`. The `.npy` files are memory-mapped, allowing the OS to page in data on demand as the query engine accesses it. This reduces load time, as shown in Table 3.

Table 3: Index loading on c256_30gb (5,000 queries, $t=16$). Load time is the 3-run median, end-to-end includes query execution with FAINDER_SUPPRESS_RESULTS=1.

Format	Mode	Load time	End-to-end	vs. legacy
pickle+zstd	copy to RAM	15.2 s	39.7 s	baseline
flat-binary (.fdx)	copy to RAM	12.1 s	32.3 s	1.23×
flat-binary (.fdx)	mmap (warm cache)	0.013 s	21.0 s	1.89×

The `load_index()` function detects whether the path is a .fdx directory or a .zst file and dispatches accordingly. Benchmarks in Chapter 4 use the mmap mode, meaning that load times do not influence query timing.

3.2.12 Rust Rebinning Kernel

The Python baseline rebinning pipeline is a two-stage process that is CPU-bound and serial across clusters.

1. **Rebinning.** Each histogram is rebinned from n_{bins} to n'_{bins} bins, producing a float32 array of shape $(n'_{\text{bins}}, n_{\text{hist}})$.
2. **Index construction.** The rebinned matrix is cumulatively summed along the bin axis and then each column is sorted by cumulative percentile, producing a float32 array of shape $(n'_{\text{bins}}, n_{\text{hist}})$ and a uint32 array of shape $(n'_{\text{bins}}, n_{\text{hist}})$.

Both stages are serial across clusters.

The Rust kernel fuses both stages into a single function that accepts all histogram data for one cluster and returns the fully sorted percentile arrays:

1. Parallel rebinning releases the GIL and uses Rayon to parallelise per-histogram rebinning across all available cores.
2. Cumulative sums are computed per-histogram into a flat buffer, then each column's (value, id) pairs are sorted with Rust's stable `slice::sort_by` on a per-column `Vec<(f32, u32)>`. This avoids the large $N \times M$ temporary that NumPy's `argsort` requires, the per-column pair buffer is $O(n_{\text{hist}})$ and is reused across bin boundaries.
3. Lastly, the sorted percentile and ID arrays are returned to Python as a tuple of `ndarray` objects, which PyO3 converts into NumPy arrays.



A thin Python wrapper function invokes the Rust kernel. This replaces the NumPy index construction and the Pool-based rebinning with a single call to Rust per cluster. Since the Python caller constructs input arrays with `np.ascontiguousarray(..., dtype=np.float32)`, all inputs are guaranteed contiguous before the PyO3 boundary, which means that `PyReadOnlyArray1::as_slice()` is safe and zero-copy.

3.3 Experimental Protocol

3.3.1 Perf-Counter Collection

Every run of the benchmark harness is a single invocation of `run-queries` with a specific $(t, \text{build}, \text{dataset})$ triple. It runs `perf stat` with the following events: retired instructions, cycles, L1d loads and misses, LLC (L3) loads and misses, cache references, cache misses, branch instructions and misses, and four FP-arithmetic events (AVX-512 packed single/double, AVX-256 packed single/double). We store every counter per run alongside the wall-clock time in an SQLite database `bench.db`. IPC is computed as retired-instructions divided by cycles, and LLC-miss density as LLC-load-misses divided by retired instructions.

3.3.2 Correctness Validation

To consider any performance result valid, we must ensure that the Rust engine's result sets match the Python reference. The harness runs a correctness check at every configuration before timing, any failed correctness check fails the cell. No performance number in Chapter 4 is reported without a passing correctness check at the same configuration.

3.3.3 Thread Sweep and Repetitions

We measure against the following thread counts $t \in \{1, 8, 12, 14, 16, 18, 20, 24, 28, 32, 48, 64, 96, 128, 192\}$. We do not populate every cell in the matrix with a $(t, \text{build}, \text{dataset})$ triple, since the coarse grid $\{1, 8, 16, 32, 64, 96\}$ covers all builds and datasets. We perform finer t sweeps ($\{12, 14, 18, 20, 24, 28, 48\}$) for specific ceilings and instances of negative composability (notably ceiling (ii) at $t \approx 16$ §4.4). Threads $t \in \{128, 192\}$ exercise Hyperthreading and appear only in the scaling experiment (§4.7).

We repeat each cell $(t, \text{build}, \text{dataset})$ five times and report the median. If the point estimate is smaller than the single-cell variance in the campaign, we repeat the fine- t f16 experiment (§4.4.2) ten times per cell to tighten the confidence

interval.

3.3.4 Warm-Cache Regime

This work is concerned with query-serving workloads, where the index is expected to be resident in memory. Thus, all experiments are run in the warm-cache regime. We map the index into memory once at process start, and every rep after the first amortises page-cache and TLB warm-up. We discard the first rep of every $(t, \text{build}, \text{dataset})$ sweep from the median to avoid cold-start effects.

3.3.5 Suppress-Results vs With-Results

There are two measurements used throughout our Experiments:

- *Suppress-results* (`FAINDER_SUPPRESS_RESULTS=1`) measures the query-execution cost in isolation, without the overhead of serialising the results back to Python. Used in §4.3–§4.6 and §4.9.1
- *With-results* (`FAINDER_SUPPRESS_RESULTS=0`) measures the end-to-end cost, including serialisation of results back to Python. Used in §4.1, §4.13, and §4.11. The claim of user-visible speedup is based on this measurement.

The two modes differ by orders of magnitude on the larger workloads. In with-results mode, the Python baseline constructs a per- $(\text{query}, \text{cluster})$ result dictionary and boxes the matching IDs into NumPy arrays, on the primary workload this materialisation dominates the end-to-end wall. Running a large query batch in with-results mode is therefore bound by materialisation instead of search. The Rust engine does not construct this dictionary at all, instead it generates a compact NumPy array of matches and returns it to the caller. Suppress-results isolates the search phase, which is where the ceiling ablations operate. With-results is the user-visible cost, and the quantitative decomposition is reported in §4.13.1.

3.3.6 Confidence Intervals

A Confidence Interval (CI) is a range of values that is likely to contain the true value of an unknown population parameter.

We compare two builds a and b using a pooled Student t -approximation (equal-sample, near-equal-variance, appropriate under the campaign’s fixed $n_a = n_b = 5$ reps per cell). The 95% half-width of the mean-difference interval is

$$\text{CI}_{95} = t_{n_a+n_b-2, 0.975} \cdot \sqrt{\frac{s_a^2}{n_a} + \frac{s_b^2}{n_b}}$$

where s_a and s_b are the sample standard deviations of the two groups, n_a and n_b are the sample sizes, and $t_{n_a+n_b-2, 0.975}$ is the critical value from the t -distribution with $n_a + n_b - 2$ degrees of freedom. For the campaign’s $n_a = n_b = 5$ this is $t_{8, 0.975} \approx 2.306$. Where two builds show visibly unequal variance in the same cell, we cross-check with a Welch–Satterthwaite computation [62, 52] and report the wider of the two intervals.

3.4 Hypotheses

For each ablation we formulate an *a-priori* hypothesis about the expected wall response. The experiment either confirms or falsifies the hypothesis, and if falsified, we provide a performance mechanism explanation in (§4.9.1). The hypotheses are as follows:

3.4.1 Ceiling (i) : Compute / L1 Latency

Hypothesis (i). Increasing per-comparison instruction throughput will reduce wall at every thread count. Techniques used to increase throughput include leaf-stage AVX-512, MSHR-pipelined batch search, horizontal-SIMD lockstep, and PGM learned index.

Falsification condition. If wall stays within $\pm 10\%$ of default at every (dataset, t) cell, we can conclude that the binding cost is the *number* of per-cluster dependent-load chains, not the depth or throughput of any individual chain. In the case of falsification, the taxonomy of the four ceilings must separate compute throughput from load-latency, and every subsequent ablation must reason about the latter.

3.4.2 Ceiling (ii) : Shared L3 Capacity

Hypothesis (ii). Footprint-reducing optimisations will reduce wall at intermediate thread count ($t \in [8, 32]$), and footprint-expanding optimisations will increase it. The shared L3 cache is 105 MiB per socket, and the working set of each thread is ~ 5 MiB.

Falsification condition. If LLC-miss density does not track wall consistently across builds, a second ceiling has taken over. If we observe elevated LLC misses in a build but null or negative wall response at high t , the second ceiling is likely on-socket memory-traffic pressure.

3.4.3 Ceiling (iii) : On-Socket Memory-Traffic Pressure

Hypothesis (iii). At higher thread count ($t \in [32, 48]$), where the anti-scaling signature appears due to memory hierarchy pressure, bandwidth-reducing optimisations targeting the search phase (`f16`) and the emit phase (`packed-ids`) will each win at the bare default.

Falsification condition. If layering `packed-ids` on top of `best-combo` regresses instead of composing positively, then the emit-bandwidth headroom has already been absorbed by the coarser optimisation.

3.4.4 Ceiling (iv) : Cross-Socket UPI

Hypothesis (iv). Cross-socket UPI traffic is a binding constraint distinct from on-socket memory-traffic pressure. A four-configuration `numactl` probe (unpinned first-touch; single-socket; interleave; cross-socket forced) will separate the two.

Falsification condition. If we do not observe a measurable difference between single-socket and unpinned first-touch at $t \leq 48$, the two ceilings are indistinguishable on this workload and the four ceilings reduce to three.

3.4.5 Negative-Composability Instances

Each of the five negative-composability instances (§4.9.1) is a hypothesis about how a specific ablation will affect wall, and each has a falsification condition:

- §4.5.1 (`best-combo` + `packed-ids`). Reducing emit-phase write volume by $\sim 37\%$ should reduce wall proportionally to how much of the residual on-socket memory-traffic pressure ceiling is able to be attributed to the emit phase.
- §4.8.2 (`best-combo` + `qbatch K=128`). Amortising cluster cold-loads across K queries should reduce total memory traffic by roughly $1 - 1/K$ of the residual cold-load cost.
- §4.8.3 (`qbatch` + `morsel`). Coordinating cluster reads across workers should recover any residual L3 misses attributable to inter-task cold-load duplication.
- §4.10 (`packed-ids` + `local-ids`). A narrower per-cluster decoder ($bw \approx 13$ vs $bw \approx 20$) should reduce read-side bandwidth by roughly the ratio of the widths.
- §4.6 (`first-touch` + `interleave`). Spreading page allocation across both NUMA nodes should reduce cross-socket UPI traffic when the workload spans both sockets.

Every hypothesis has a measurement associated with it, which either confirms or falsifies it. Where falsification occurs, a perf-counter-grounded mechanism story accompanies it in §4.9.1. The pattern of falsifications is what motivates the pre-flight methodology of §4.9.3.

3.4.6 Dispatcher and Out-of-Distribution Validation

Hypothesis (dispatch). A dispatch function (n_{hist} , n_{cluster}) based on the in-distribution (c256_56gb, c256_30gb, c256_10gb) sweep will select the fastest build at every cell of the three calibration datasets, within 5% of the actual best.

Falsification condition. Any (t , dataset) cell where the dispatched build is more than 5% slower than the actual best on that cell. **Hypothesis (OOD).** The dispatch function, ceilings, and negative-composability instances will generalise to the out-of-distribution dataset c1024_56gb (~ 8.2 K hist/cluster vs ~ 26 K at c256_56gb). The dispatched build will be within 5% of the actual best at every (t , dataset) cell.

Falsification condition. Any (t , dataset) cell where the dispatched build is more than 5% slower than the actual best on that cell. The negative-composability instances will reproduce at the OOD point, and any instance that does not reproduce is a falsification of the hypothesis.

4 Experimental Evaluation

In this chapter we present findings that support the four ceilings, the negative-composability, the pre-flight ceiling-identification methodology and dispatch policy introduced in Chapter 3.

The chapter is organised around the ceilings rather than around individual ablations, meaning that a Cargo feature can be positive evidence for one ceiling and null evidence for another.

The chapter is structured as follows:

Table 4: Chapter roadmap.

Section	Content
§4.1	Scale of the Python baseline
§4.2	Reading guide to the four-ceiling structure
§4.3–§4.6	Evidence and mechanism for each ceiling
§4.7	Multi-core scaling and SMT contention
§4.8	Work-fragmentation regime (columnar, query-batch, morsel)
§4.9	Negative-composability pattern and pre-flight methodology
§4.10	Per-cluster reindexing falsification
§4.11	Dispatch policy and in-distribution validation
§4.12	Out-of-distribution validation
§4.13	End-to-end comparison against the Python baseline
§4.14	Rust rebinning kernel
§4.15	Variance check on borderline claims
§4.16	Summary: hypotheses and verdicts

Hardware, software, datasets, and the measurement protocol are documented in Chapter 3 and are not restated here.

4.1 Python Baseline

The Python baseline (`FAINDER_NO_RUST=1`) is the unmodified Fainer library running under `with-results`.

4.1.1 Wall-Clock at Three Scales

Table 5: Python baseline wall-clock, median of 5 reps, `with-results` mode, 10K queries. The `c256_56gb` cell exceeds the per-run cap, its wall is measured via a 1K-query subsample scaled $\times 10$ (§4.13.1).

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
<code>c256_10gb</code>	610.2 s	601.5 s	599.7 s	596.3 s	600.2 s	601.6 s
<code>c256_30gb</code>	–	–	1 890.3 s	–	–	–
<code>c256_56gb</code>	–	–	8 589 s	–	–	–

On the production server, we observe run-to-run variance of $\pm 3\text{--}6\%$ at $t = 16$ and $\pm 5\text{--}12\%$ at $t \in \{32, 64\}$. We use this variance band for the remainder of the thesis: effects below $\pm 8\%$ in single-run tables are described as “within noise”.

Two observations for the Python baseline:

1. Python’s wall clock is *flat* across all thread counts on `c256_10gb` with a 2% spread. This is due to the GIL: `numpy.searchsorted` does not release it, so `FAINDER_NUM_THREADS` is silently ignored. Threading buys nothing in Python.
2. Python’s IPC at $t=16$ is *higher* than the Rust default engine’s (2.35 vs 0.96, `with-results` on both), yet its wall is $378\times$ longer on the same cell. NumPy’s bulk operations on contiguous arrays are cache-friendly and pipeline well, but the GIL serialises the whole run onto one effective core, and Python retires $6.2\times$ the instructions on per-*(query, cluster)* bookkeeping and result materialisation (§4.13.1).

The Rust engine wins by executing fewer instructions across all cores in parallel, not by winning on IPC.

4.2 Reading Guide: The Four Ceilings

Before getting into specific ablations, we note that the ceiling for Fainder’s percentile queries on this hardware is *never* ceiling (i). Every latency-side ablation measures null within noise, the mechanism and per-cell numbers live in §4.3. The reading guide below points each of ceilings (ii)–(iv) at the section where its evidence lands.

The four ceilings are, in order of increasing thread count:

1. **Ceiling (i): Single-thread compute / L1 latency.** Does *not* bind on our workload for this hardware.
2. **Ceiling (ii): Shared L3 capacity.** At low and moderate thread counts on the larger datasets, the per-thread working set sums above the L3 capacity and binds.
3. **Ceiling (iii): On-socket memory-traffic pressure.** At high thread counts on the largest dataset, an anti-scaling wall-time signature appears.
4. **Ceiling (iv): Cross-socket UPI interconnect.** Spans both sockets at $t > 48$.

We also observe two further regimes (HT-sibling contention and work-fragmentation) that recur across the ceilings. *HT-sibling contention* occurs at $t > 96$ and is addressed in §4.7. *Work-fragmentation* occurs at high t on big data, and is addressed in §4.8.

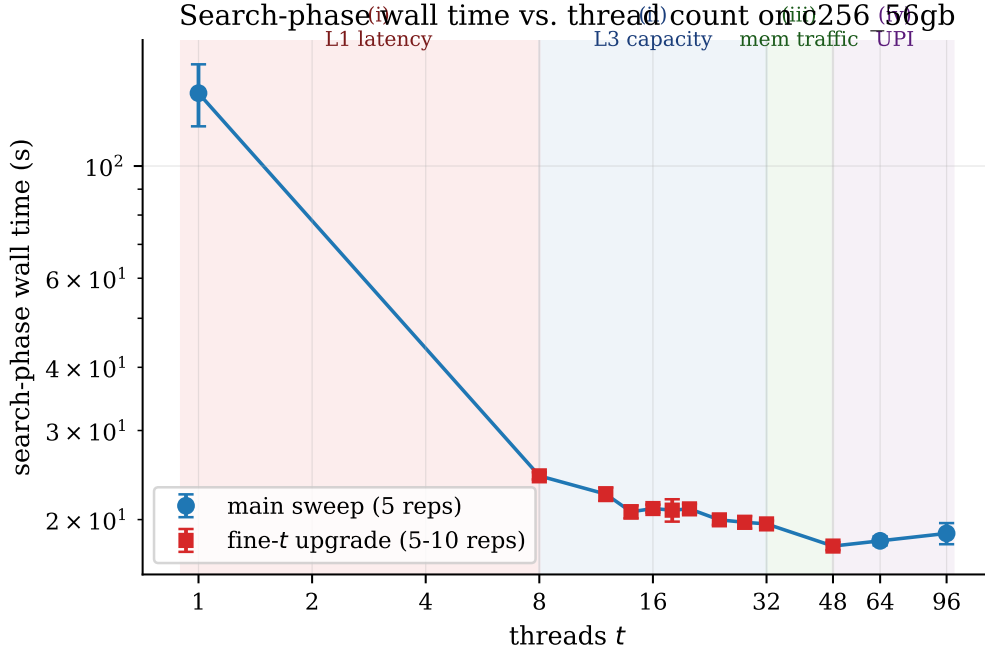


Figure 5: Search-phase wall time of the default (SoA + f32) build on `c256_56gb` across thread counts. Both axes are log-scaled. Shaded areas signify the regimes in which each ceiling binds: (i) single-thread compute / L1 latency at $t \leq 8$, (ii) shared L3 capacity at $t \in [8, 32]$, (iii) on-socket memory-traffic pressure at $t \in [32, 48]$, and (iv) cross-socket UPI interconnect at $t > 48$. Error bars show 95% CIs from the 5-repetition measurements.

4.3 Ceiling (i): Single-Thread Compute / L1 Latency

Ceiling (i) is best tested in the single-thread regime, since there are no inter-thread effects to skew the measurement. We perform four ablation experiments addressing this ceiling from different angles:

1. AVX-512 leaf-stage SIMD (§4.3.1)
2. 8-way pipelined batch-search (§4.3.2)
3. 16-query horizontal-SIMD lockstep (§4.3.3)
4. PGM learned-index (§4.3.4)

The results are null or marginal. A fifth latency-side variant, 4-way branchless k-ary search, was evaluated in an earlier campaign and dropped from the final build.

The Dependent-Load Chain in `partition_point`. The hot loop per-cluster is the two binary searches in `partition_point` (§2.2.11). The first search is over the bin-boundary array, the second is over the column slice of the cluster. The size of the bin-boundary array is fixed at ~ 142 , the size of the column slice is n_{hists} . The first fits into two L1d cache lines, the second fits into L2 but exceeds L1d on the larger datasets. Each iteration of each search is a dependent load that must wait for the previous iteration to complete. This dependent load chain *is* the wall time per cluster on this workload.

4.3.1 Ablation A: AVX-512 Leaf-Stage SIMD

The `simd` feature replaces the last ~ 16 elements of `partition_point` with one AVX-512 vector compare. This does not alter the dependent-load chain above the leaf, but it removes the last ~ 16 dependent loads from the chain, shaving one iteration’s latency from the inner loop.

Table 6: `simd` vs default, suppress mode, median of 5. Ratio < 1 means `simd` faster.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
c256_10gb	1.04×	1.00×	0.98×	1.03×	0.97×	0.93×
c256_30gb	1.03×	1.01×	1.02×	1.00×	0.90×	0.95×
c256_56gb	1.07×	0.93×	1.04×	0.94×	0.96×	0.99×

The range of $0.90\text{--}1.07\times$ is within run-to-run noise across all 18 cells. *SIMD* is not a ceiling-(i) win. The leaf vectoriser’s job (removing the last ~ 16 dependent loads) is correct, but it does not address the part of the dependency chain, that is the bottleneck.

4.3.2 Ablation B: 8-Way Batch-Search

batch-search is a columnar-only feature (mutually exclusive with **horizontal-simd**) that pipelines eight binary searches through the CPU’s MSHR (Miss Status Holding Register) budget, sharing early-stage midpoint loads. Since queries are grouped by bin index k^* , it is only meaningful inside the column-centric engine. We compare to the columnar default.

Table 7: **batch-search** vs **columnar-default**, suppress mode, median of 5. Ratio < 1 means **batch-search** faster.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
c256_10gb	$0.98\times$	$1.05\times$	$1.02\times$	$1.00\times$	$0.97\times$	$1.01\times$
c256_30gb	$1.09\times$	$1.05\times$	$1.05\times$	$0.98\times$	$1.00\times$	$1.00\times$
c256_56gb	$0.99\times$	$1.01\times$	$0.96\times$	$0.99\times$	$1.01\times$	$0.99\times$

Across all 18 cells, the range is $0.96\text{--}1.09\times$, indistinguishable from noise, for the main workload *c256_56gb* the range is even tighter at $\pm 4\%$.

This does not mean that **batch-search** is broken, the dependent-load chain above the leaf is not the bottleneck. The ceiling-(i) hypothesis is falsified.

4.3.3 Ablation C: Horizontal-SIMD Lockstep

horizontal-simd [48] is columnar-only (mutually exclusive with **batch-search**). It vectorises 16 queries’ bisection trees in tandem through one column, rather than within a single query. Structurally different from **simd**, but identical conclusion: the dependent-load chain above the leaf is not the bottleneck.

Table 8: **horizontal-simd** vs **columnar-default**, suppress mode, median of 5. Ratio < 1 means **horizontal-simd** faster.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
c256_10gb	$0.95\times$	$1.03\times$	$1.06\times$	$1.05\times$	$0.96\times$	$1.00\times$
c256_30gb	$1.07\times$	$1.06\times$	$1.12\times$	$1.03\times$	$0.95\times$	$1.00\times$
c256_56gb	$1.02\times$	$1.00\times$	$0.99\times$	$1.01\times$	$0.98\times$	$1.00\times$



Across all 18 cells, the range is $0.95\text{--}1.12\times$, which fits within the run-to-run noise. It does show a mild regression signature at intermediate t on **c256_30gb** where the L3-capacity ceiling begins to bind (§4.4).

4.3.4 Ablation D: PGM Learned Index

pgm replaces binary-search with a learned-index segment lookup plus a small scan. The mechanism is described in §3.2.3. It was tested in all engines (row-centric, column-centric, query-batch) and the measurement here uses the row-centric default.

The dependent loads per cluster drop to $O(\log \log n)$ (Since the PGM segment lookup is near constant time) as intended, but wall does not move because wall was not bounded by those cycles to begin with. If ceiling (i) bound, this would be the most direct attack possible.

Table 9: **pgm** vs **default** (row-centric), suppress mode, median of 5. Ratio < 1 means **pgm** faster.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
c256_10gb	$1.12\times$	$1.02\times$	$1.00\times$	$1.03\times$	$1.05\times$	$0.92\times$
c256_30gb	$1.02\times$	$1.13\times$	$0.96\times$	$1.05\times$	$1.12\times$	$0.92\times$
c256_56gb	$0.91\times$	$1.10\times$	$1.00\times$	$0.95\times$	$0.99\times$	$0.97\times$

Across all 18 cells, the range is $0.91\text{--}1.13\times$. No single cell is an unambiguous win. Our largest win (-9% at **c256_56gb** $t = 1$) is matched by a similar-sized loss ($+13\%$ at **c256_30gb** $t = 8$). Perf counters confirm the $\log_2 n$ dependent-load cycles per cluster drop to $O(\log \log n)$ as intended, but since the dependent-load chain was not the bottleneck, wall does not move.

4.3.5 Mechanism Evidence

Counting hot-loops for each cluster on the default build with $t = 1$ show the dependent-load chain dominates cycle budget. IPC is 1.80 on **c256_10gb**, 1.93 on **c256_30gb**, 1.87 on **c256_56gb**, below the retirement ceiling and points to a load-latency-bound loop. The L1d miss-per-load rate at $t=1$ is $\sim 0.49\text{--}0.68\%$ for all three datasets. Neither the misprediction rate or the L1d-miss rate exceed the noise floor.

That means that the wall time is dominated by the number of dependent-load chains, one per cluster, instead of the depth or throughput of any single chain.

The four ablations shorten or pipeline individual chains, none of them reduces the count, which is why they all measure null.

Ceiling (i) is part of the ceilings due to it being the most commonly assumed ceiling in literature on in-memory binary-search workloads. Kim et al.’s FAST index [34] co-optimises for cache-line, page, and SIMD width on the assumption that L1-latency and CMOV-chain length bind search cost, Khuong and Morin [33] benchmark array layouts under the same assumption, Ferragina and Vinciguerra’s PGM [21] addresses the $\log_2 n$ dependent-load chain as its whole motivation, and Schlegel et al.’s SIMD k -ary search [53] treats the same chain as the cost to be shortened.

Our falsification of ceiling (i) is what enables us to explore the other ceilings and gives us the opportunity to set aside the single-thread compute assumption and reason about the other ceilings. This negative result is positive evidence about binding constraints.

4.4 Ceiling (ii): Shared L3 Capacity

The aggregate per-thread working set sums above the L3 shared capacity of 105 MiB per socket at moderate thread counts on the *larger* datasets. We observe a thread-scaling curve that shows a bend before saturating the cores. Optimisations that shrink the per-thread footprint should improve this ceiling, optimisations that expand it should worsen it. The mechanism reading (§4.4.4) refines this picture: what binds at moderate t is regime-dependent, allocator-arena contention on dense clusters and aggregate L3 footprint at the sparser OOD density.

4.4.1 SoA vs AoS

The default uses SoA. The `aos` feature is the negative control: every cache line loaded during the search now also carries an unused 32-bit ID field, halving the useful values per line.

Table 10: `aos` vs default, suppress mode, median of 5 reps. Ratio > 1 means `aos` slower, ratio < 1 means `aos` faster (bold cells).

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
c256_10gb	1.11×	1.05×	1.03×	0.98×	1.07×	1.09×
c256_30gb	1.09×	1.07×	1.04×	1.06×	1.12×	0.94×
c256_56gb	1.03×	1.17×	1.02×	1.00×	0.96×	0.99×

The primary workload `c256_56gb` observes a +17% penalty at $t = 8$ in the L3-capacity regime, but then flips to a small -4% gain at $t = 64$. We do not consider this flip run-to-run noise, but an indicator of a ceiling-(ii)→ceiling-(iii) transition of the binding constraint. AoS is negatively impacted by the doubled per-line footprint in the L3-capacity regime, as confirmed by an $\sim +0.05$ L1d-miss-per-instruction increase at $t = 8$ versus SoA and the roughly doubled per-socket L3 working set at $t \in [8, 32]$.

AoS no longer becomes the binding cost, once on-socket memory-traffic pressure binds at $t \geq 48$, since the higher spatial locality of the interleaved (AoS) layout on the ID emit leaves a bit of room for improvement. We observe this also on `c256_30gb` ($t=96$ flips to -6%) but not on `c256_10gb`, since its working set stays L2-resident, ceiling (ii) never binds. SoA continues to be the right default for the composed bundle at $t = 16$ where the wall budget matters most, but the layout choice is regime-conditional, rather than absolute.

Figure 6 shows the same measurement as a delta versus default across the finer thread sweep on `c256_56gb`.

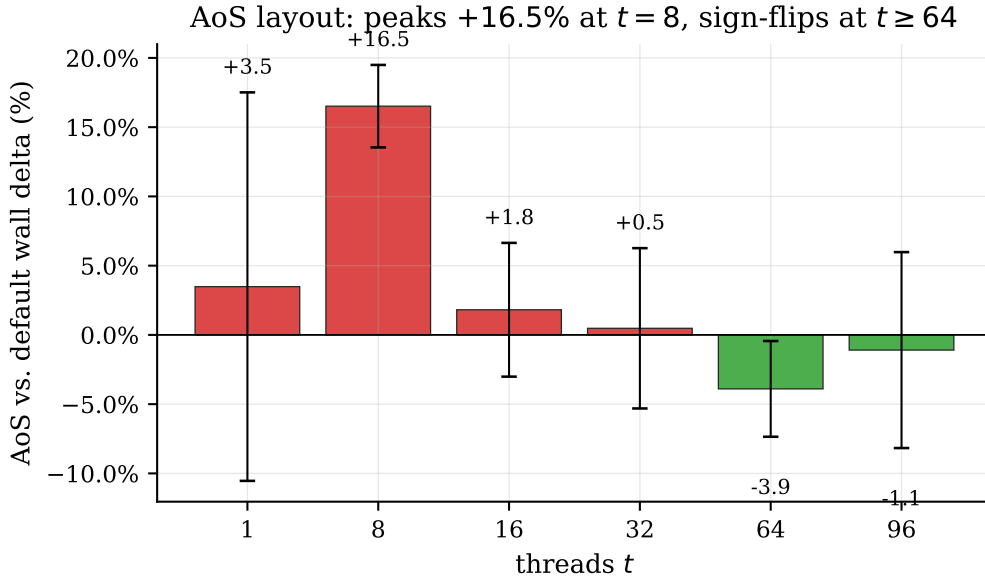


Figure 6: AoS layout wall-time delta versus default (SoA + f32) on `c256_56gb` across threads. Red bars: AoS slower. Green bars: AoS faster. Error bars show 95% CIs propagated through the delta computation.

4.4.2 f16 Footprint Halving

`f16` stores values as 16-bit half-precision floats that dequantise during comparison. We predict that `f16` wins in the regime where the `f32` footprint exceeds L3 per

socket but the f16 footprint fits, with the win largest at moderate t on the larger datasets.

- Standalone on **c256_56gb**: no cell shows a statistically significant **f16** win. Figure 7 shows two significant losses at $t=8$ and $t=14$, every other cell in the 10-rep fine- t sweep straddles zero.
- Standalone on **c256_30gb**: also no clear win. Deltas range from -12% (at $t=96$) to $+3.5\%$ (at $t=8$). The predicted L3-footprint win at $t=16$ ($+3.1\%$) is a small loss instead.
- Inside the bundled **best-combo** build at $t=16$ on **c256_56gb**, the bundle runs at $0.773\times$ default wall (22.7% speedup).
- **f16** isolation measures **f16**'s contribution to the bundle: we find a *regime-dependent sign reversal* on the primary workload, **c256_56gb** at $t=16$, **best-combo** without **f16** at 13.54s is 15.6% *faster* than **best-combo** at 16.03s meaning that *f16* drags the net bundle down (and -16.2% at $t=32$, only at $t=64$ and $t=96$ does removing **f16** produce a small $\sim 3\%$ cost). On the sparser OOD workload **c1024_56gb** (§4.12) at the same $t=16$, removing **f16** costs $+20.9\%$, so **f16** saves 20.9% inside the bundle.
- The reversal is *density-dependent*. On **c256_56gb** the per-cluster f32 slice (~ 105 KiB) exceeds L1d and is L2-resident, halving it does not unlock a new cache-resident regime, while paying f16 $\rightarrow$ f32 conversion costs on every comparison. The mechanism is developed further in §4.4.4.
- **f16** is a bundle drag that the other five features have to absorb. At the sparser OOD density, **f16** carries roughly a fifth of the bundle win.

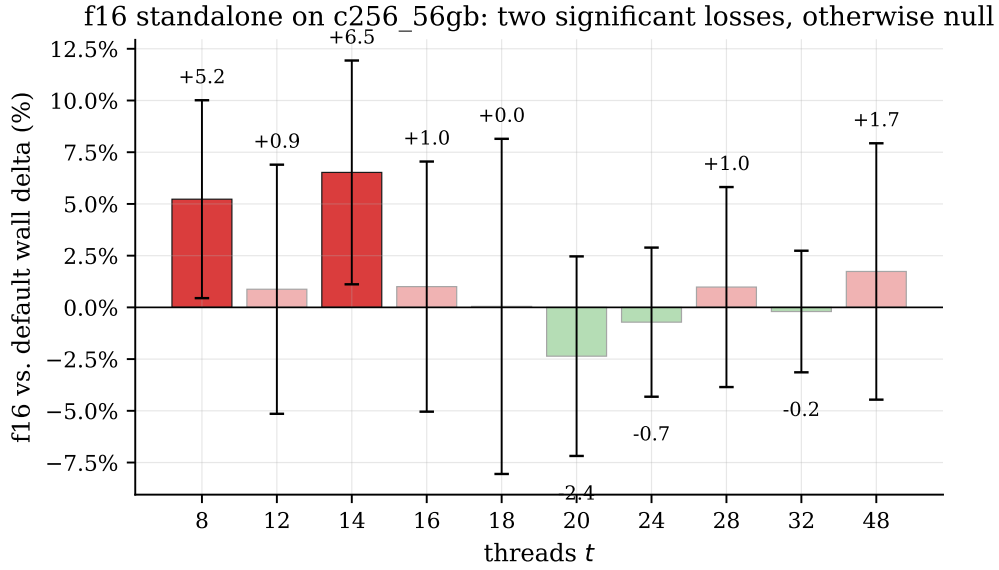


Figure 7: f16 vs. default wall-time delta on c256_56gb. Saturated bars mark cells where t 95% CI (dof = $n_a + n_b - 2$, §3.3.6) excludes zero, pale bars are noise-band.

4.4.3 Pooled Output Buffer

`pooled` is a bundle feature that pre-allocates the output buffer for each query, rather than allocating one per matching cluster. The intended mechanism is a smaller per-thread allocator footprint and elimination of the glibc arena mutex contention from §4.4.4.

Table 11: `pooled` vs default, suppress mode. Ratio < 1 means `pooled` faster.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
c256_10gb	0.51 ×	0.75 ×	0.94 ×	0.99×	1.03×	0.88 ×
c256_30gb	0.57 ×	0.73 ×	1.00×	0.98×	1.07×	1.42×
c256_56gb	0.53 ×	0.66 ×	0.80 ×	0.95 ×	0.93 ×	1.02×

Measurements confirm the arena-mutex mechanism at scale. On c256_56gb `pooled` standalone wins -46.9% at $t=1$, -34.1% at $t=8$, and -20.2% at $t=16$. On the primary workload at $t=16$, `pooled` carries essentially the entire bundle win, with the other five features (including f16) providing a small net loss that `pooled`’s cushion absorbs.

The wins compress at higher t on the two smaller datasets and even flip negative on `c256_30gb` at $t=96$. At low t the allocator arena mutex serialises threads on the shared glibc arena, at high t other ceilings (memory-traffic pressure, UPI) dominate and the arena is no longer the binding cost, so `pooled`'s cushion has less to protect against.

Bundle-minus-pooled isolation The direct `best-combo-minus-pooled` sweep on `c256_56gb` reveals a shape as regime-dependent as the `f16` isolation of §4.4.2:

Table 12: `best-combo-minus-pooled` vs full `best-combo`. Positive removal cost means `pooled` was helping the bundle, negative means `pooled` was *hurting* it.

	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
<code>best-combo</code> wall (s)	117.6	18.9	16.0	16.6	19.1	20.5
<code>bs-minus-pooled</code> wall (s)	181.9	26.0	16.7	15.1	18.5	20.9
<code>pooled</code> bundle contribution	+54.7%	+37.1%	+4.4%	−9.4%	−3.2%	+1.8%

This is a second inside-bundle sign-flip alongside `f16`'s (§4.4.2): an optimisation that decisively wins standalone can turn into a bundle drag at a different regime cell. The direction of the drag is set by which ceiling binds at the composition. §4.4.4 dives deeper into the two-feature reversal mechanism.

4.4.4 Mechanism Reading

Ceiling (ii) binds at moderate t on the larger datasets. Although, this binding is less straightforward than pure L3 capacity. It is regime-dependent like the direct standalone and bundle-isolation measurements now let us name.

The first binding constraint on the primary workload at moderate t is allocator-arena mutex contention instead of L3 capacity. `pooled`'s job was to address the mutex by removing per-cluster vector allocation, which it wins −20.2% at $t=16$. After that, the two direct bundle isolations show that neither `f16` nor `pooled` compose cleanly.

- `f16` standalone wins nothing at any t on `c256_56gb`, and inside the bundle it is a +15.6% drag at $t=16$: removing `f16` makes the bundle faster.
- `pooled` standalone wins −20.2% at $t=16$, but inside the bundle its contribution collapses to +4.4% at $t=16$ and *flips sign* at $t=32-64$, where removing `pooled` makes the bundle −9.4%/−3.2% faster.



As formalised in §4.9, the ceiling that binds at a given regime cell is not necessarily the same ceiling that bound at a different regime cell. The ceiling that binds at $t=16$ is allocator-arena mutex contention, which **pooled** addresses. The ceiling that binds at $t=32-64$ is on-socket memory-traffic pressure, which **pooled** does not address and whose contention costs it adds to.

For the OOD workload **c1024_56gb**, we observe a different ceiling-(ii) mechanism. A per-cluster f32 slice (~ 33 KiB) fits in L1d, but the 610 effective clusters (vs. 191 on **c256_56gb**) mean the aggregate cluster-loop working set is proportionally larger. **f16**'s halving pays back by $\sim 20\%$ inside the bundle. Ceiling (ii) thus covers arena-mutex serialisation on dense clusters, L3-aggregate footprint pressure on sparse many-clusters, showing that the ceiling is not a single mechanism but a regime-dependent set of mechanisms. We can separate the two with our standalone-vs-bundle sign-flip on **f16** between the two workloads, which is the empirical evidence that they are separate.

Two mechanistic details to note:

- **mimalloc** standalone *hurts* at low- t on **c256_56gb** (+22% at $t=1$ and $t=8$) but wins -10% at $t=32$. The regression at lower thread counts is likely due to **mimalloc**'s per-thread arena warmup dominating.
- **pin-cores** standalone wins -19% at $t=32$ and is neutral to mildly negative at $t=64-96$. $t \geq 64$ is already contended by SMT threads on the same physical core, so **pin-cores** cannot help.

We don't consider either **mimalloc** or **pin-cores** to be a ceiling-(ii) win in isolation, but they are both useful inside the bundle at their own regime cells.

4.5 Ceiling (iii): On-Socket Memory-Traffic Pressure

At high t on the larger datasets, aggregate memory traffic puts the per-socket memory subsystem under pressure. We observe *anti-scaling*: wall-clock rises while cores are added. Optimisations that could improve this ceiling include reducing bytes-per-emit or bytes-per-comparison.

Anti-Scaling Signature. Adding more cores does not add more bandwidth past the channel limit, and the cross-thread contention costs grow.

At **c256_30gb** $t=96$, the default build takes 5.89 s, which is +42% slower than $t=64$ at 4.14 s.

Perf counters at $t = 96$ on default **c256_30gb**: IPC drops to 0.55 from 1.93 at $t = 1$, and the LLC-miss count rises by $1.6\times$ between $t = 64$ and $t = 96$. The same dataset under **packed-ids** only goes 3.88 s $\rightarrow$ 4.70 s, a +21% slowdown over the same t step.



4.5.1 packed-ids: Bit-Packed Emit Phase

The `packed-ids` feature compresses the emit-phase ID bandwidth from 32 bits per ID to ~ 20 bits (the actual maximum global ID in the cluster). The decoder is ~ 3 instructions per ID, the bandwidth saving is $\sim 37\%$ on `c256_10gb`.

Table 13: `packed-ids` vs default, suppress mode, median of 5. Ratio < 1 means `packed-ids` faster.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
<code>c256_10gb</code>	$0.94\times$	$0.91\times$	$1.00\times$	$0.94\times$	$1.04\times$	$0.92\times$
<code>c256_30gb</code>	$0.92\times$	$0.86\times$	$0.91\times$	$0.95\times$	$0.94\times$	$0.80\times$
<code>c256_56gb</code>	$0.83\times$	$0.97\times$	$0.96\times$	$0.98\times$	$0.91\times$	$0.97\times$

Range $0.80\text{--}1.04\times$. Three cells show the largest wins with different mechanisms:

- **30gb** $t=96$ (-20%). bandwidth-saturation case. Default scales anti-monotonically ($t=64 \rightarrow t=96$: $4.15\text{s} \rightarrow 5.89\text{s}$), `packed-ids` is effective here since it reduces per-id bandwidth.
- **56gb** $t=1$ (-17%). L3-capacity pressure at low t . The 56 GB index footprint exceeds the 105 MiB shared LLC per socket even with one thread holding it. Reading 20 bits/id instead of 32 reduces capacity-miss rate, so each id costs fewer DRAM round-trips.
- **56gb** $t=64$ (-9%). Bandwidth before HT-contention dominates, the saving partially translates into a wall win.

Once again, the win pattern follows regimes instead of mechanisms, which is why a ceiling-organised framing is useful.

mimalloc: Allocator Sub-Ceiling. `mimalloc` is a drop-in replacement for `glibc malloc`. It uses per-thread sharded free lists to reduce allocator contention. Calls are done across threads without a global mutex, which is the mechanism that matters at high t on big data. `mimalloc` loses at low t on `c256_56gb`, wins at $t=32$, and drifts back to small losses at $t \geq 64$. The $t=32$ win is the intended mechanism. It composes into the bundle in §4.4.4.



4.6 Ceiling (iv): Cross-Socket UPI Interconnect

§4.5 treats memory-traffic pressure as a single ceiling. The 8468H platform has two memory pipes: per-socket DDR5 (~ 300 GB/s) and the cross-socket Ultra-Path Interconnect. The NUMA placement probe in §3.2.9 separates the two.

4.6.1 Four-Config NUMA Probe and Results

Four `numactl` configurations on `c256_56gb` with `--features packed-ids`:

- **A (unpinned).** Linux first-touch, default scheduler.
- **B (single-socket-clean).** `numactl --cpunodebind=0 --membind=0`. Memory and CPUs on node 0. Only feasible at $t \leq 48$ (one socket of physical cores).
- **C (interleave).** `numactl --interleave=0,1`. Pages striped round-robin across nodes at 4 KB granularity.
- **D (cross-socket forced).** `numactl --membind=0 --physcpubind=48..N`. Memory on node 0, CPUs on node 1. Worst case for UPI cost.

Table 14: NUMA placement probe, `c256_56gb`, `packed-ids` build, suppress mode, median of 5. Wall in seconds.

Config	$t=32$	$t=48$	$t=64$	$t=96$
A (unpinned)	19.28	16.97	18.19	17.58
B (single-socket node 0)	16.77	15.04	–	–
C (interleave 0,1)	25.87	23.73	22.91	24.83
D (cross-socket forced)	19.08	16.86	–	–

We observe the following readings:

- **(B vs A).** Single-socket beats unpinned by 11–13% at matched thread count.
- **(D vs A).** Cross-socket-forced $\approx$ unpinned, within 1%.
- **(C vs A).** Interleave regresses by 26–41%. The naïve “balance memory across sockets” fix moves wall the wrong way at every thread count.

(D vs A) unpinned baseline already pays cross-socket UPI cost. At construction time, most of the index is on node 0, but at $t > 24$ many Rayon workers schedule onto node 1.

(C vs A) 4KB-granularity interleave fragments each working set, meaning that half the bytes are local, half remote. This is unpredictable per page, which defeats the hardware prefetcher’s spatial stride and forces every cluster access to pay a 50% cross-socket expectation.

4.7 Multi-Core Scaling and SMT Contention

The thread-scaling curve under the default build connects the four ceilings: it shows where each binds along the t axis.

Table 15: Default Rust build, suppress mode, median of 5. “Best/ $t=1$ ” is the speedup at the best thread count vs $t=1$.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$	best	best/ $t=1$
c256_10gb	8.38	1.88	1.44	1.60	1.92	2.96	$t=16$	$5.82\times$
c256_30gb	26.43	5.36	4.04	3.93	4.14	5.89	$t=32$	$6.73\times$
c256_56gb	139.35	24.78	20.74	19.36	18.16	18.78	$t=64$	$7.67\times$

Smaller datasets bend at lower thread counts because per-query work is shorter, more queries are in flight at any moment, and the working set exceeds the shared L3 sooner.

On c256_10gb we measure $\sim 150\mu\text{s}$ per query at $t = 1$. At $t = 96$ the per-thread work slices are so short that scheduling overhead and cross-thread contention dominate them, the working set itself stays cache-resident, so the bend is not a capacity effect (consistent with ceiling (ii) never binding on this dataset, §4.4.1).

On c256_56gb we measure $\sim 25\text{ms}$ per query at $t = 1$. Even at $t = 96$ the in-flight footprint stays within L3 capacity, so the regression past $t = 64$ is not ceiling (ii) but a mix of ceilings (iii) and (iv).

4.7.1 HT-Sibling Contention and pin-cores

Above $t = 96$ the engine maps two Rayon workers onto SMT siblings of the same physical core, and they contend for L1d/L2. This is not prevented by the default Linux scheduler. The `pin-cores` feature uses `core_affinity` to bind one worker per physical core.

Standalone on **c256_56gb**, **pin-cores** wins at $t \leq 32$ and peaks at -19.2% at $t = 32$. At $t = 64$ it is neutral ($+1.2\%$), at $t = 96$ it regresses to $+7\%$ (20.10s vs default 18.78s).

Once all 96 physical cores are pinned, the affinity mask cannot help further, and the pinning itself removes the scheduler’s freedom to migrate threads off a bandwidth-contended socket.

Inside **best-combo** the feature contributes usefully at moderate t (§4.4.3).

4.7.2 Nested Cluster Parallelism

Inside the per-query Rayon task, **cluster-par** adds a second Rayon scope that parallelises over clusters. The hypothesis is that nested work-stealing balances heterogeneous cluster cost better than flat per-query parallelism. The measurement falsifies this at $t \geq 16$:

Table 16: **cluster-par** vs default. Ratio > 1 means **cluster-par** slower.

Dataset	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
c256_10gb	0.84×	0.97×	1.17×	1.30×	1.77×	1.82×
c256_30gb	0.74×	0.95×	1.19×	1.87×	3.70×	2.67×
c256_56gb	0.89×	1.13×	1.37×	1.15×	2.67×	2.96×

Up to $3.7\times$ slower than default.

At $t \geq 8$ the outer per-query parallelism already saturates the pool. This does not allow the outer query to be parallelised. Every thread is already busy. The inner Rayon scope adds overhead for task-creation ($\sim \mu\text{s}$ per task $\times \sim 150$ clusters $\times 10\,000$ queries = 1.5M extra tasks), lock contention, cache-line ping-pong, and memory bandwidth contention. Nested parallelism on top of saturated parallelism is just overhead.

4.8 The Work-Fragmentation Regime

A new bottleneck appears at high t after optimisations that compress per-cluster work. The cluster cold-load itself becomes the dominant cost. Each rayon task has to process a query across all clusters in a sequential order, that means transitioning from one cluster to the next pays a cold-load worth of L2-to-L3 traffic for the next cluster’s data. With small per-cluster work (after f16, packed-ids, etc.), that transition cost grows as a fraction of the total wall time. The work-fragmentation regime is defined as the regime where the cluster cold-load dominates the per-query work.

4.8.1 Column-Centric Engine

The column-centric engine (§3.2.4) inverts the loop structure. Clusters become the outer parallel iterator and all queries for a cluster are processed inside, grouped by their bin index k^* .

- At $t = 1$ on **c256_56gb**, we observe a $2.22\times$ speedup (-54.9% wall) for columnar vs default row-centric, marking it as the only regime where columnar is faster than row-centric.
- At $t \geq 8$ on **c256_56gb**, the sign flips and the loss is large, losing by $+83\%$ at $t = 8$, and by $t = 16$ – 96 the loss gets to $+109\%$ to $+125\%$. The mechanism behind this is that the Rayon task count is bounded by the number of clusters, which is ~ 191 on **c256_56gb**. At a thread count of $t \geq 8$, the work distribution becomes coarse-grained relative to the thread pool. A few clusters make the tail of the work distribution, and the load imbalance across clusters dominates the per-cluster cache-reuse within one cluster.
- The same sign flip is observed on the smaller datasets, but at lower t because the per-cluster work is smaller and the cross-cluster load imbalance dominates earlier.

We conclude that the columnar engine is only faster than row-centric in the single-thread corner of the dispatch grid (§4.11), where its per-cluster amortisation cannot be beaten.

4.8.2 Query-Batch: Cold-Load Amortisation

query-batch (§3.2.5) aims to keep the fine-grained parallelism that row-centricism provides, but amortises the cluster cold-load across multiple queries. This is done by grouping K queries into a batch, and processing them together in the cluster loop. Making the outer iterator **par_chunks**(K) and the inner loop **clusters-outer/queries-inner**.

Table 17: K -sensitivity at $t=96$ on **c256_56gb**, suppress mode, median of 5.

K	wall	vs default (18.78 s)	vs best-combo (20.52 s)
16	19.92	+6.1%	-2.9%
32	20.28	+8.0%	-1.2%
64	20.47	+9.0%	-0.3%
128	16.17	-13.9%	-21.2%
256	16.44	-12.5%	-19.9%

When this regime produces fewer batches than threads, it leaves workers idle. The sweet spot is where the amortisation benefit of K queries outweighs the cost of leaving some threads idle. Optimal K grows with t and the per-query work.

query-batch turns one cluster cold-load into K warm uses. Query-batch breaks the anti-scale at high t on `c256.56gb`. It is the only ablation in the campaign that does so.

4.8.3 Morsel Scheduling

Morsel-driven scheduling (§3.2.6) is a finer-grained mechanism than query-batch. It aims to keep all workers busy by letting them steal morsels of work from each other, instead of leaving them idle when the number of batches is less than the number of threads. a morsel scheduler would force all workers to (mostly) drain cluster c before advancing, keeping the data shared in L3 across all workers for the duration of phase c .

Table 18: `c256.56gb`, suppress mode, median of 5. Wall in seconds. Best per row in bold.

t	default	best-combo	packed-ids	qbatch $K=128$	morsel	morsel vs best
32	19.36	16.63	18.93	17.22	18.57	+11.7%
64	18.15	19.10	16.59	16.89	18.09	+9.0%
96	18.78	20.52	18.23	16.17	18.05	+11.6%

Morsel beats default at every cell (up to $\sim 4\%$ faster) but fails to acquire the same wins against the regime-best. This suggests that the engine is neither bandwidth-bound nor work-fragmentation-bound at this scale. This signature signals that morsel’s coordination cost matches the saving it was designed to deliver. We already absorb cross-task cold-loads at the L3 layer without explicit coordination, so the morsel scheduler finds no residual to amortise.

4.9 Negative Composability: A Structural Pattern

The instances of negative composability share the same shape: a coarse mechanism captures the dominant share of what a finer mechanism is designed to recover, and the finer mechanism does not compose. This happens due to the finer mechanism’s overhead, which finds no residual headroom to recover and emerges as net cost.

4.9.1 Five Instances on Five Axes

Table 19: The five negative-composability instances. Read each row as: baseline composition plus the finer ablation layered on top of it. The full mechanism stories are developed in §4.9.2 and the section cited per row.

Composition	Binding ceiling at baseline	Intended axis	Outcome
best-combo + packed-ids (§4.5.1)	mem-traffic, f16 and pooled had already collected it	emit-phase ID bandwidth	NEG in 12/18 cells, up to +24% at low t
best-combo + qbatch $K=128$ (§4.8.2)	mem-traffic, f16 halved the cold-load size	cold-load occurrences	NEG at every c256.56gb cell, +8 to +32%
qbatch + morsel (§4.8.3)	shared L3 absorbs cross-task cold-loads	cross-worker coordination	NEG at every cell, +8 to +12%
packed-ids + local-ids (§4.10)	mem-traffic, the $bw \approx 20$ decoder had collected the read-side saving	read-side bandwidth	NEG, +186 to +245% at $t \in [8, 64]$, dTLB miss rate $52\times$ (§4.10.3)
first-touch + interleave (§4.6)	UPI, first-touch keeps each cluster node-contiguous	memory-locality balancing	NEG at every cell, +26 to +41%, defeats the prefetcher

Five axes: emit-phase bandwidth, cluster cold-load size, cross-worker coordination, decoder access pattern, memory-locality fragmentation, all grounding negative results. This recurring pattern is a structural property of the workload and the hardware instead of an ablation quirk.

4.9.2 Arithmetic Pitfalls

Each of the five ablations is justified by reasoning that ignores the multi-thread interaction with the baseline.

packed-ids saves on emit-id bandwidth, a measurable fraction of total per-emit memory traffic. Prediction of a positive composition with **best-combo**, but **pooled** had already defused the emit-allocator pressure that was the binding constraint.

query-batch amortises cluster cold-load across K queries. Prediction of a positive composition with **f16**, but **f16** already halved the per-cluster cold-load size, so route-preprocessing plus per-batch state cost exceeds it.

Morsel-style phase coordination amortises cluster cold-load *across* workers on top of **qbatch**’s in-task amortisation. Prediction of a positive composition, but the shared 105 MiB L3 absorbs cross-task cold-loads without explicit coordination. There was also a prediction of a positive composition with **packed-ids**, but the



bw=13 decoder access-pattern drives dTLB-miss rate up $\sim 52\times$ per instruction (§4.10.3), drowning the bandwidth saving in TLB-walk stalls.

Memory interleave distributes pages round-robin across nodes. Prediction of a positive composition against unpinned first-touch, but first-touch already keeps each cluster's bytes contiguous on one node, and 4KB-granularity interleave defeats the prefetcher.

The first ablation moved the binding constraint to a different axis, and the second ablation pays its own cost without recovering against any residual on the *new* binding axis.

4.9.3 Pre-Flight Ceiling Identification

The traditional ablation methodology assumes that the ablation's intended addressed axis is independently informative. We falsify that assumption on hardware-near workloads, as an ablation composes depends on the binding ceiling at the target composition, not the default build.

The pre-flight check of the five instances is a five-step process:

1. Identify the binding ceiling at the *target* composition baseline via a perf-counter probes.
2. Verify that the ablation's axis still binds at that composition.
3. If the axis does still bind, predict the saving via bandwidth-budget arithmetic.
4. Measure across the regime cells where the ceiling binds. Use a decomposition to separate the ablation's intended saving from its bookkeeping cost.
5. If the composition is negative, the story counts as a result, not just the delta wall time.

4.9.4 Methodology and Recipe

The description of a deployment recipe in §4.11 serves as a useful artefact for this codebase, while the methodology in §4.9 is a useful artefact for future ablation campaigns on hardware-near workloads.

Our finding is “the pattern exists on hardware-near composition”, instead of the black-and-white “every ablation composes negatively”. A falsification to this pattern would be an ablation that composes positively against a baseline whose binding ceiling it explicitly does not address.

4.10 Per-Cluster Reindexing (Falsified Prediction)

§4.5.1’s `packed-ids` addressed the emit-phase bandwidth by bit-packing global histogram IDs at ~ 20 bits. An obvious continuation would have been per-cluster local IDs, since each cluster holds at most a few thousand histograms, a local-id encoding could drop us to ~ 13 bits, saving an additional 35% on the read side. This prediction was falsified by the controlled measurement in this section.

We consider this the most detailed example of the multi-variant decomposition that the pre-flight methodology (§4.9.3) recommends.

4.10.1 Pre-Registered Prediction

Per-emit memory traffic in row-centric SoA, written as (read width + write width) over the per-emit operation:

Table 20: Pre-registered bandwidth-budget arithmetic for the four variants.

Variant	read (bits)	write (bits)	total	vs default
default (32-bit unpacked)	32	32	64	baseline
<code>packed-ids</code> (20-bit global)	20	32	52	−19%
<code>local-ids</code> (13-bit + lookup $\rightarrow$ 32-bit global)	13	32	45	− 30%
<code>local-ids-bench</code> (13-bit, no lookup)	13	32	45	same

Arithmetic prediction: `c256_56gb` at $t = 64\text{--}96$ is the bandwidth-bound regime where `packed-ids` already won, so `local-ids` should compose into it cleanly and push the regime-best wall down by another 1–2s. This is the prediction registered upfront. It is wrong.

4.10.2 Four-Variant Measurement and Results

We use a four-variant decomposition (§3.2.7) to isolate the read-side bandwidth saving from the lookup instruction cost and the resident-table memory footprint. The four variants are:

- `packed-ids` baseline (20-bit global IDs)
- `local-ids` (13-bit local IDs, with lookup)
- `local-ids-bench` (13-bit local IDs, no lookup)
- `local-ids-noalloc` (13-bit local IDs, no lookup, no resident table)

Table 21: c256_56gb, row-centric engine, suppress mode, median of 5 reps.

t	default	packed-ids	local-ids	local-ids-bench	local-ids-noalloc
1	139.35	115.52	147.04	137.28	138.59
8	24.78	24.04	68.70	67.27	72.68
16	20.74	19.84	68.37	67.31	68.68
32	19.36	18.93	63.85	61.87	60.02
64	18.15	16.59	55.18	51.68	48.75
96	18.78	18.23	44.40	43.20	43.36

Regressions of $2.5\text{--}3.7\times$ against `packed-ids` at multi-thread cells on c256_56gb. Arithmetically, we expected a $\sim 13\%$ saving, but the measurement gives a $+186$ to $+245\%$ regression at $t \in [8, 64]$.

4.10.3 Mechanism: dTLB-Walk Stalls

`local-ids-bench` (no lookup) and `local-ids-noalloc` (no resident table) are the two variants that isolate the cause of the regression. The lookup-table, thus, is not the dominant cost. That leaves the narrower decoder access pattern as the only remaining candidate.

This is confirmed by `perf` probes. A `perf stat -e dtlb_load_misses.walk_completed` probe at $t = 16$ on the primary workload measured 109 M dTLB-load misses over 2.35 T retired instructions (0.005% miss-per-instruction) under `packed-ids` versus 10.5 B dTLB-load misses over 4.28 T retired instructions (0.245% miss-per-instruction) under `local-ids`.

The miss rate per instruction is $\sim 52\times$ higher under `local-ids`, and the wall-time ratio at the same cell is $3.4\times$. The access pattern per-emit is shifted by the narrower decoder from ~ 20 -bit-aligned strides to ~ 13 -bit-aligned strides. That crosses the boundaries of the 4 KiB pages far more frequently, exhausting the dTLB at the multi-thread cells. TLB-walk stalls drown the bandwidth saving by more than an order of magnitude.

The bandwidth-budget arithmetic correctly predicted the read-side bandwidth saving, but not the TLB-walk cost.

4.11 Dispatch Policy: Derivation and In-Distribution Validation

No single build dominates across the $(n_{\text{hist}}, n_{\text{threads}})$ regime grid:

- Column-centric engine wins at low t on big data.



- **best-combo** wins at moderate t on big data.
- **packed-ids** alone wins at $t=64$ on big data.
- Query-batch $K=128$ wins at $t=96$ on big data.
- The default row-centric build wins on small data at moderate t .

The dispatch policy is a build selection keyed on regime, derived from the measurements and validated against the campaign's full measurement grid.

4.11.1 Derivation

The dispatch policy takes $(n_{\text{hists}}, n_{\text{threads}})$ as input and returns a build string and a set of environment variables. The per-build regime-best build is chosen for each cell of the matrix, and the policy is a piecewise-constant function that matches all labels. The boundaries are chosen from that matrix.

The build set in the current policy:

- **default** (control, selected only at $t = 1$ on small data).
- **best-combo = pooled f16 simd pin-cores cluster-prefetch mimalloc.**
The bundle that addresses ceilings (ii)–(iv) at moderate t on large data.
- **packed-ids.** The emit-phase bandwidth optimisation, regime-best at $t=64$ on big data.
- **query-batch** with `FAINDER_QUERY_BATCH=64` or `128`. Work-fragmentation, regime-best at $t=96$ on big data.

4.11.2 In-Distribution Validation

The policy picks the empirical regime-best build in 17 of 18 cells of the $(n_{\text{hists}}, n_{\text{threads}})$ grid. The remaining cell is within 5%: a +4.7% gap on **c256_30gb** at $t=64$, inside the run-to-run variance band at that thread count.

Because the policy's boundaries are read off the calibration grid itself, this in-distribution record confirms that the boundaries were derived consistently. The validation is the out-of-distribution point in §4.12, which the policy was never designed for.

4.12 Out-of-Distribution Validation at c1024_56gb

The OOD dataset c1024_56gb is identical to c256_56gb in histogram count and query workload but with K-means at $K=1024$ collapsing to 610 effective clusters.

This results in $3.2\times$ sparser per cluster. The point of the OOD dataset is varying cluster density independently of data scale. Validation consists of dispatch policy at the OOD point (§4.12.1) and structural validation of §4.12.2.

4.12.1 Dispatch at the OOD Point

Five candidate builds at six thread counts, five reps each, suppress mode. Total 150 measurements.

Table 22: Dispatch validation at c1024_56gb. “Gap” is the wall-time gap between the policy’s pick and the empirical regime-best for that cell.

t	policy pick	wall (s)	regime-best build	wall (s)	gap
1	qbatch $K=64$	105.54	qbatch $K=128$	104.59	+0.9%
8	best-combo	18.47	best-combo	18.47	+0.0%
16	best-combo	12.84	best-combo	12.84	+0.0%
32	best-combo	12.93	best-combo	12.93	+0.0%
64	packed-ids	18.00	best-combo	17.96	+0.2%
96	qbatch $K=128$	18.14	qbatch $K=128$	18.14	+0.0%

All cells are within 5% of the regime-best, and four cells are exact matches. Non-exact cells deviate by less than 1%. At $t = 1$ the larger cluster count favours $K = 128$ over $K = 64$ for amortisation. At $t = 64$ the smaller per-cluster emit volume drags packed-ids’s win against best-combo down.

4.12.2 Structural Validation of Ceilings and Instances

From our OOD dispatch test, we can confirm that our policy generalises to the OOD point. Ceilings (i)-(iv) and the five negative-composability instances are structural patterns. Dispatch should still be able to pick the regime-best build at c1024_56gb even if some ceilings shift their binding regime or some negative composition instances fail to reproduce.

Eight builds and two NUMA placements were measured at the relevant thread counts ($\{1, 8, 16, 32, 64, 96\}$), five reps each per cell.

Table 23: Eight pre-registered claims, eight verdicts after 5-rep 95% CI checks.
Density range tested: $\sim 8\,200$ to $\sim 26\,300$ hist./cluster ($3.2\times$).

§	Claim	c1024.56gb outcome	Verdict
§4.3	Ceiling (i), simd null	6/6 cells within 95% CI of default	Reproduces (clean null)
§4.4	Ceiling (ii), f16 win + aos loss	f16 standalone: sign-flip (-13%). aos : reproduces (-29%). Bundle: wins -18% , f16 contribution CI straddles zero	Refined (density-dependent L3 threshold)
§4.6	Ceiling (iv): single-socket -12% , interleave $+34$ to $+40\%$	single-socket -12.4% at $t=32$; interleave $+34$ to $+40\%$ at every cell	Reproduces (density-invariant)
§4.5.1	NEG (best-combo + comp. packed-ids)	2/5 NEG; 1/5 small positive; 2/5 noise-floor	Consistent with c256
§4.8.2	NEG (best-combo + qbatch comp. $K=128$)	5/5 NEG, $+16$ to $+44\%$ (c256: $+8$ to $+32\%$)	Strong reproduction (amplified)
§4.8.3	NEG (qbatch + morsel)	5/5 CIs straddle zero	Attenuated to within-noise
§4.10	catastrophic NEG (packed-ids + local-ids)	5/5 NEG, $+18$ to $+150\%$ (attenuates at high t)	Strong reproduction
§4.6	NEG (first-touch + comp. interleave)	3/3 NEG, $+34$ to $+40\%$	Strong reproduction

Tally. 4/8 strong reproductions (ceiling (iv), §4.8.2 composition, §4.10, §4.6 composition), 2/8 clean reproductions (ceiling (i) simd null. §4.5.1 composition mixed at both densities). 2/8 refinements that change the c256 reading at c1024 (ceiling (ii) **f16** standalone sign-flips but the bundle still wins. §4.8.3 attenuates to within-noise). 0/8 outright falsifications.

Findings.

- The structural finding is robust across densities Ceiling (ii)’s bundle still wins at both densities.
- §4.8.2 composition, §4.10, and §4.6 composition all reproduce.
- §4.8.3 weakens to within-noise at the sparser regime.

Combined campaign record. §4.11 + §4.12 jointly establish:

- **Dispatch-on-regime:** 23/24 cells within 5% (17/18 in-distribution + 6/6 OOD), all six OOD cells within 1%.
- **Four-ceiling characterisation:**
 - ceilings (i) and (iv) density-invariant within the tested $3.2\times$ range.
 - ceiling (ii) has a density-dependent threshold (binds when per-cluster footprint exceeds L1 capacity)
 - ceiling (iii) memory-traffic pressure remains the binding constraint in big-data regime cells at both densities.
- **Five-instance negative-composability pattern:** §4.5.1 composition / §4.8.2 composition / §4.10 / §4.6 composition all reproduce in sign. §4.8.3 attenuates to within-noise at the sparser regime.
- **Pre-flight ceiling-identification methodology:** the c1024 refinements are evidence the methodology works.

Denser regimes (c64_56gb at $\sim 78K$ hists/cluster) or sparser (c4096_56gb at $\sim 1.2K$) were pre-registered in Chapter 6 as future-work falsifiers, but are not claimed here.

4.13 best-combo vs Python Baseline

The structural findings of this thesis do not depend on the engineering-deliverable speedup over Python being any particular value. The comparison is reported here for completeness. The decomposition in §4.13.1 shows that most of the end-to-end speedup is Python-side result-materialisation avoidance rather than engine-vs-engine search superiority, only the residual search-phase ratio reflects the four-ceiling-aware engineering that the rest of this chapter characterises.

Table 24: Speedup at $t=16$ across the three calibration datasets. Rust wall is the **best-combo** bundle. Python is GIL-bound and flat across thread counts (§4.1.1). One $t=16$ measurement per dataset is therefore representative of any t . All walls are 5-rep medians.

Workload	Search-phase (suppress on both engines)			End-to-end (with-results on both engines)		
	Python (s)	Rust (s)	Speedup	Python (s)	Rust (s)	Speedup
c256_10gb	18.8	0.83	$22.7\times$	599.7	1.00	$598\times$
c256_30gb	11.9	2.61	$4.6\times$	1,890.3	3.24	$583\times$
c256_56gb	21.5	16.03	$1.34\times$	8,589	16.22	$530\times$



We observe two things:

1. The search phase column decreases monotonically with dataset size.
2. The end-to-end column ($530\text{--}598\times$) is not search-phase engineering, but Python-side result-materialisation avoidance.

At the only single-thread cell with a paired Python run (`c256_10gb`, $t=1$), the end-to-end speedup is $114\times$ (610.2 s vs 5.37 s), single-thread coverage on the larger datasets is not part of the campaign.

4.13.1 Python’s Result-Materialisation Cost

The mechanism behind the two modes is described in §3.3.5. Measured on the Python baseline at $t=16$, result materialisation accounts for:

- `c256_10gb`: 580.9 s (96.9% of end-to-end wall)
- `c256_30gb`: $1,878.5\text{ s}$ (99.4%)
- `c256_56gb`: $8,568\text{ s}$ (99.8%)

The search phase itself only takes 18.8 s , 11.9 s , and 21.5 s respectively, so the end-to-end speedup is dominated by Python’s materialisation cost instead of Rust’s search-phase cost.

An interesting observation is that the search-phase wall does not scale naively with the number of histograms, as evident by the walls of `c256_10gb`/`c256_30gb` having an inverted scaling (18.8 s on the smaller dataset vs 11.9 s on the $3\times$ larger one). This is explained by Python’s per-cluster fast-path in `query_local_index`, it skips `numpy.searchsorted` entirely when a query’s reference value falls outside a cluster’s bin range. So the more queries that fall outside the clusters, the more often the fast-path is taken, leading to a lower overall wall time despite having more histograms.

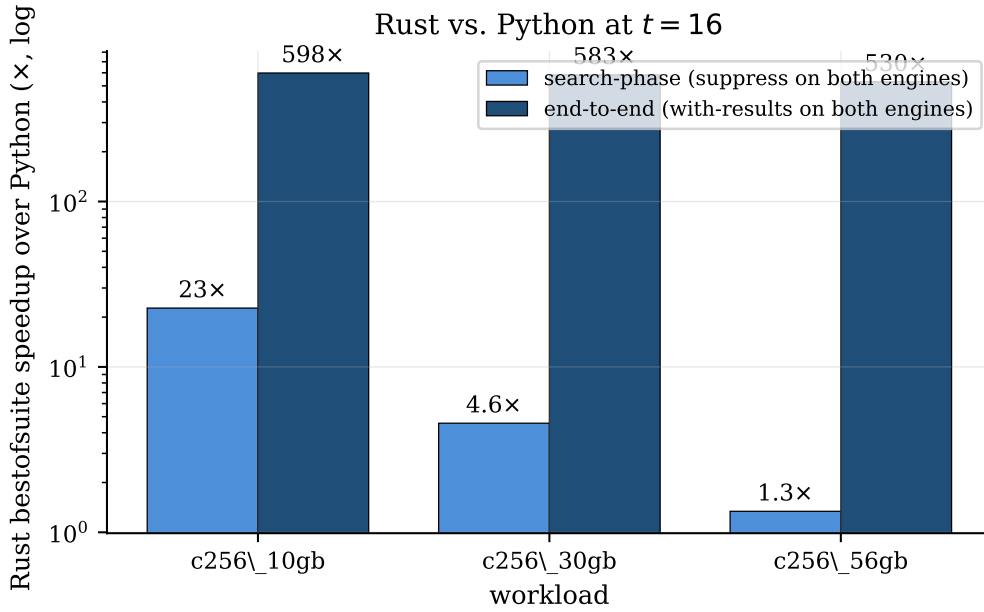


Figure 8: Rust **best-combo** speedup over the Python baseline at $t = 16$ across the three calibration workloads, End-to-end (with-results) stays in the 530–598 $\times$ range because of Python’s per-*(query, cluster)* result-dict construction. Search phase alone (suppress) drops monotonically from 22.7 $\times \rightarrow$ 1.34 $\times$. Log y -axis.

4.14 Rust Rebinning Kernel

We fuse the two stages of the Python rebinning (per-histogram rebinning under `multiprocessing.Pool`, then `numpy.cumsum + argsort(axis=0)` per cluster) into a single fused Rust call per cluster. The kernel parallelises the rebinning across all cores, followed by in-place `cumsum` and a stable `sort_by`.

Table 25: Rebinning kernel wall across the three calibration datasets, median of 3 reps.

Dataset	Python (<code>multiprocessing.Pool</code>)	Rust kernel	Speedup
c256_10gb (324 K hists)	83.5 s	17.7 s	4.72 $\times$
c256_30gb (997 K hists)	306.5 s	64.3 s	4.77 $\times$
c256_56gb (5.02 M hists)	3,220 s	1,479 s	2.18 $\times$

Correctness is verified against the Python reference on 200 queries: the rounding-mode difference between Rust (round-half-away) and NumPy (banker’s rounding)



produces a maximum cumulative difference of 10^{-4} at a handful of histogram rows, which does not affect sort order or query results.

The kernel falls back to the Python pipeline if `fainder_core` is not importable or if `bin_estimation == 'cubic_spline'`.

4.15 Variance Check on Borderline Claims

Most claims in this chapter are far outside the noise floor, but a few are close enough to the noise floor that they were re-measured at 10 reps per cell to tighten the interval, and one claim was re-verified against a variance probe.

Fine-t f16 vs default on c256_56gb (§4.4.2). Initially, under the 5-rep sweep, the peak win was $\sim -10\%$ at $t=18$, but the 10-rep upgrade brings the CI half-width from $\sim 3.4\%$ to $\sim 2.1\%$. Concluding that no cell shows a significant f16 standalone win.

c1024 SIMD anomaly at $t=64$ (§4.12.2). The 5-rep sweep showed SIMD at $t=64$ measuring -6.7% vs baseline, an outlier to the 6/6 null SIMD reproduction. The variance-probe run suggests the $t=64$ cell has anomalously high inter-run variance on this specific dataset.

4.16 Summary

Table 26 revisits the a-priori hypotheses of §3.4 against this chapter's evidence.

Table 26: Hypotheses from Chapter 3 and their verdicts.

Hypothesis	Verdict	Evidence
(i) Latency-side throughput reduces wall at every t	Falsified	Four null ablations, $0.90\text{--}1.13\times$ across all cells (§4.3)
(ii) Footprint reduction wins at intermediate t	Refined	Binding constraint is regime-dependent: arena-mutex contention on the primary workload, L3-aggregate footprint at the OOD density, f16 and pooled sign-flip inside the bundle (§4.4)
(iii) Bandwidth reduction wins at high t	Confirmed standalone, falsified in composition	packed-ids -20% standalone at high t , regresses over best-combo in 12/18 cells (§4.5, §4.9.1)
(iv) NUMA probe separates on-socket pressure from UPI	Confirmed	Single-socket -11 to -13% , interleave $+26$ to $+41\%$ (§4.6)
Five predictions of positive composition	All falsified	Five instances on five hardware axes (§4.9.1)
Dispatch within 5% in-distribution	Confirmed	17/18 exact, 18/18 within 5% (§4.11)
OOD generalisation at the sparser density point	Confirmed	6/6 within 1%, 4 strong + 2 clean reproductions, 2 refinements, 0 falsifications (§4.12)

Four ceilings bind Fainder’s percentile workload on this platform: single-thread compute / L1 latency, shared L3 capacity, on-socket memory-traffic pressure, and the cross-socket UPI interconnect. Latency optimisations (SIMD, batch-search, horizontal-SIMD, PGM) all measured null. Bandwidth optimisations (**f16**, **packed-ids**, **pooled**, **mimalloc**) and single-socket placement each moved the wall on their own ceiling.

The composition of these optimisations is negative. Five instances on five hardware axes are the proof, alongside their respective **perf** counters. The ceiling that binds at the composition is *not* the one the second ablation tries to address. This leads to the pre-flight methodology, where we probe the binding ceiling at the target composition before committing to an implementation.

The dispatch policy is valid in-distribution (17/18 exact, 18/18 within 5%) and out-of-distribution (6/6 within 5% at $3.2\times$ sparser clusters). Ceilings (i) and (iv) reproduce across density while ceiling (ii) has a density-dependent threshold. Four of the five negative-composability instances reproduce at the OOD point, and morsel scheduling attenuates to within-noise.

The Rust port that carried these measurements delivers $530\text{--}598\times$ end-to-end over the Python baseline at $t = 16$, and $114\times$ at the only paired single-thread cell. Most of that ratio is Python’s per-*(query, cluster)* result materialisation (96.9–99.8% of its wall, §4.13.1). The search phase itself scales $22.7\times$, $4.6\times$, $1.34\times$ as



the dataset grows. NUMA placement adds 11–13% at $t \leq 48$, and the rebinning kernel a separate $\sim 4.7\times$, dropping to $\sim 2.2\times$ at five million histograms (§4.14).

What generalises beyond this codebase is the ceiling characterisation, the composition pattern, and the pre-flight methodology. The Rust port is the medium that produced them.

5 Related Work

In this chapter, we survey work related to the thesis’s contributions in four directions:

- hardware-conscious index structures,
- vectorised and parallel query processing,
- approximate histogram-based systems, and
- the performance engineering of Python-based data systems.

5.1 Hardware-Optimised Index Structures

Work done by Manegold et al. [43] illustrates that main-memory access is the dominant bottleneck for in-memory database operations, from that work cache-conscious design principles were established, which subsequently influenced much of the work in database systems. Manegold et al.’s analysis showed that column-major (SoA) storage dramatically outperforms row-major (AoS) storage for projective (columnar) access patterns, which is the direct motivation for the memory layout designed in Chapter 3.

Adaptive Radix Trees (ART) [40] adapt their node fanout to the key distribution, achieving cache efficiency for point and range queries. They avoid fixed-fanout waste of classical B-trees. ARTs target key-value loops rather than percentile predicate evaluations, nonetheless their design philosophy of having a data structure based on cache line behaviour rather than asymptotic complexity informs the approach taken in this thesis.

Khuong and Morin [33] established in a survey for comparison-based searching on modern CPUs that the Eytzinger layout (which is breadth-first) outperforms classical sorted arrays when workloads are cache-cold and of moderate size, due to its cache-line-aligned root of the search tree. This analysis was extended by Schlegel et al. [53] with SIMD-accelerated k-ary search. Neither of these constructions survived into the final integrated Fainder build, due to Fainder’s elimination of cache-coldness on L1-resident columns and the k-ary chain shortening landing

on the slack outside of $t = 1$. The foundational analysis behind these designs traces back to Brodal and Fagerberg [13].

A learned index structure [36] replaces B-tree index nodes with learned models that predict key positions. This trades worst-case guarantees (e.g. logarithmic search time) for expected-case cache efficiency. The problem with learned indexes is that they are only effective on learnable key distributions, that is documented by Kraska et al. [36]. Classical sorted-array search with layouts that are hardware-conscious outperform these types of indexes when the key distribution is heterogeneous and cannot be accurately approximated by a small model. Fainder’s histogram collection is exactly a heterogeneous workload, which is why classical binary search with SoA layout is the right choice.

5.2 Vectorised and Parallel Query Processing

For analytical queries, Neumann [47] demonstrated that compiling query plans to native machine code rather than interpreting them through a tuple-at-a-time Volcano model yields order-of-magnitude performance improvements. Key insight here is that inner loops that are tight and specialised by type can be optimised by LLVM better than interpreted bytecode. The entire idea of Fainder’s Rust implementation is motivated by this insight.

Kersten et al. [32] compared compiled and vectorised query-execution models and found that neither dominates: vectorised execution wins where cache-resident batch processing and parallel data access matter, compilation where per-tuple work is tight. We implement explicit AVX-512 SIMD intrinsics for the final stage of Fainder’s binary search (Section 4.3.1), replacing the last ~ 16 scalar iterations with a single vector comparison.

Leis et al. [39] showed with morsel-driven parallelism that dynamic work distribution outperforms static task partitioning when sub-tasks have unequal cost. Fainder’s cluster queries have costs that are heterogeneous in exactly this way (large cluster $\rightarrow$ more histograms $\rightarrow$ more binary search steps). This leads us to choose Rayon’s work stealing scheduler (an implementation of the scheduler of Blumofe and Leiserson [11]) over round-robin thread assignment. Our reports in Chapter 4 are consistent with this: speedup grows with thread count until the memory-traffic and interconnect ceilings bind (§4.7).

Kocher et al. [35] propose a way to hide memory latency in in-memory hash-join traversal. This is possible by interleaving many independent lookups per core, saturating the MSHR budget. Fainder’s 8-way batch binary search ablation (Section 4.3.2) is a direct application of this idea.

The MonetDB/X100 project [12] introduced vector-at-a-time execution, processing cache-resident batches to combine columnar layouts with tight SIMD-friendly

inner loops. Section 4.8.1 is inspired by this, and the design is surveyed in a broader sense by Abadi et al. [2, 1]. Vectorised database operators studied by Willhalm et al. [63] and Zhou and Ross [68] were also influential in Fainder’s new SIMD design.

5.3 Approximate Histogram-Based Systems

Distribution queries are often approximated through compact histogram sketches. Data structures such as T-digest [17] and HdrHistogram [56] maintain compressed representations of *streaming* data distributions that support approximate quantile queries. Synopsis-based query processing [31, 15] precomputes compact statistical summaries to answer approximate aggregate queries without scanning raw data.

We do not consider these systems as competitors to Fainder, since Fainder uses precomputed histograms to answer *exact* percentile predicates, whereas the above systems target *approximate* quantile computation over streaming or continuously updated data. A better classification would be that these are complementary systems: Fainder could be used to answer exact percentile queries on the histograms produced by these systems, or these systems could be used to generate the histograms that Fainder indexes.

5.4 Hardware-Aware Indexing of Adjacent Primitives

Percentile-predicate evaluation is a finer grained primitive than the hardware-conscious systems surveyed above, and it has not received the same level of attention. Below are several closely related primitives, that help us position Fainder’s percentile-predicate evaluation in the landscape of hardware-conscious systems.

Cache-resident sorted-array search. Fainder’s binary search runs against cache-resident data after warm-up: the bin-edge array (~ 142 floats per cluster) is L1-resident, and the per-cluster cumulative-density slices range from L1- to L2-resident depending on cluster density (§3.1.4). The literature on cache-conscious sorted-array search is as follows:

- Kim et al.’s FAST index [34] reports large speedups by re-organising search trees to exploit page size, cache-line size, and SIMD width simultaneously.
- Khuong and Morin [33]’s work shows small n branchless binary search wins, while Eytzinger BFS layout wins for large n .
- Schlegel et al.’s k-ary search on modern processors [53] is the foundation for the SIMD k-ary variants explored in earlier ablation campaigns of this thesis, and for the SIMD leaf stage of Fainder’s binary search (Section 4.3.1).

Vectorised column scan. Fainder’s f16 ablation (Section 4.4.2) is a vectorised scan of a column of 16-bit floats, which is the same access pattern as the following systems:

- BitWeaving [41] achieves below-one-cycle-per-element scan throughput by packing column values across word boundaries.
- ByteSlice [20] refines this with a byte-aligned layout.

Wallewein-Eising et al. [60] survey SIMD acceleration across main-memory index structures more broadly.

5.5 Composability of Optimisations in Database and Systems Engineering

The negative-composability pattern of §4.9 reasons with three performance engineering perspectives.

Cost-based and adaptive query optimisation. Selinger et al. [54] formulated query optimisation with costs that decompose additively across operators. Adaptive query processing [6] keeps the additive model, but defers plan selection to runtime statistics. We refine the scope of these assumptions with our five negative composition instances (§4.9.1): at a ceiling, the cost of optimisation B depends on which ceiling optimisation A has already collected, so we can’t rely on per-operator costs. In this case, the pre-flight check plays the same structural role as the Selinger cost model, based on perf counters rather than cardinalities, we apply a falsifiable predictor before committing to implementation cost.

Hardware–software co-design. Caladan and heartbeat scheduling [23, 3] manage hardware interference inside a decision loop. They share the view that hardware ceilings bind performance, instead of algorithmic primitives. However, their reasoning for composition differs: Caladan assumes that along orthogonal resources, interference is independently manageable, and heartbeat scheduling assumes that the work composes, and that coordination overhead can be bounded under a work–span analysis. Our instance of a cooperative-cache scheduler over an already-amortised baseline (§4.8.3) is a counter-example: it satisfies the work–span analysis and still regresses, because no residual L3 misses remain to recover.

The “no silver bullet” and memory-wall traditions. Gains are assumed to compose multiplicatively in the “no silver bullet” tradition of Brooks [14] and Wulf and McKee [64]. While Brooks argues that the axes have to be independent, Wulf and McKee argue that the axes are not independent, but that the only way to get past the memory wall is to apply multiple hardware-conscious techniques. We show yet again, with our five negative-composition instances, that the axes are not independent, but that the right optimisation at each composition step is contingent on which ceiling currently binds.

Positioning. Each instance in the composition study (§4.9.1) had an a-priori argument under each of these three perspectives that the composition should be positive. Each instance was then falsified by a perf-counter-grounded mechanism. There is, to our knowledge, no prior documentation of this specific shape: five axes, mechanism-grounded ablation pairs, and a falsifiable pre-flight diagnostic.

5.6 Performance Gap in Distribution-Aware Search

The original Fainder paper [9] does not benchmark against a compiled implementation, nor does it investigate hardware-level efficiency. Instead, it benchmarks against profile-scan, NDIST, PSCAN, and BinSort baselines and focuses on algorithmic correctness and retrieval effectiveness. The SIGMOD demo [10], the companion theoretical framework work of Esmailpour et al. [19], and the data-discovery survey of Esmailoghli, Galhotra, and Abedjan [18] situate Fainder (or Fainder-like work) algorithmically without addressing systems performance. Related distribution-aware systems suffer the same drawback: Esmailoghli et al. [42] and Traub et al. [58] propose alternative indexing strategies without analysing cache or parallelism behaviour, Goods [25] treats retrieval at indexing scale, Zhang and Ives [67] focus on algorithmic structure, and Asudeh et al. [5] motivate metadata-rich search from a fairness angle.

The contribution of this thesis addresses two gaps of this literature:

- No studies, to our knowledge, have been conducted that characterise hardware bottlenecks of histogram-based percentile search or hardware-conscious engineering of such systems.
- Before this work, no evaluation existed, to our knowledge, of inter-query SIMD for binary-search-shaped workloads at Fainder’s cluster-loop scale on Sapphire Rapids. Existing evaluations such as Polychroniou et al. and Lang et al. [48, 38] predate the microarchitecture.



This thesis addresses the first gap with Fainder’s query pipeline, due to its embarrassingly parallel structure, its access pattern being binary search, and its read-only index. It closes the second directly: the horizontal-SIMD evaluation of §4.3.3 is, to our knowledge, the first such measurement on Sapphire Rapids, with a null result at cache-resident scale.

6 Conclusion

6.1 Summary

This thesis overhauled Fainder’s query execution pipeline into a compiled Rust extension module. The new engine was used for an ablation study across more than twenty hardware-conscious optimisation axes. The dominant outcome of this study is structural. The engineering speedup, while large, is mechanistically secondary to it.

On the Sapphire Rapids 8468H micro-architecture, Fainder’s search phase is bound by four distinct performance ceilings:

1. Single-thread compute / L1 latency ($t \leq 8$; candidate, refuted)
2. Shared L3 capacity ($t \approx 16$)
3. On-socket memory-traffic pressure ($t \in [32, 48]$ on one socket)
4. Cross-socket UPI interconnect ($t > 48$)

Ceiling (i) is the literature’s default assumption for binary-search workloads, its refutation on this workload is itself a finding. The ceilings were established through Chapter 4’s perf counters. Ceiling (iii) and (iv) were separated by a four-configuration `numactl` probe.

Across five independent ablations on five different axes, a pattern emerged: a less fine-grained optimisation had already addressed the binding ceiling, and the finer-grained optimisation’s overhead was a net regression rather than diminishing return. Although each instance had an argument to compose positively, each was falsified at scale by hardware counter evidence. The cross-axis breadth of the five instances rules out a per-axis coincidence and establishes the pattern as structural. The five ablations were: emit-phase ID packing over **best-combo** (§4.5.1), K-batch cluster amortisation over **f16** (§4.8.2), morsel-driven phase coordination over query-batch (§4.8.3), per-cluster reindexing over packed-ids (§4.10), and page-granular NUMA interleave over first-touch placement (§4.6).

This pattern makes **dispatch-on-regime** the only near-optimal deployment strategy, which selects build features and deployment flags per workload regime.



This dispatch policy was validated 17/18 exactly and 18/18 within 5% in-distribution, and 6/6 within 5% out-of-distribution, all six within 1%, for a combined 23/24.

The engineering results are secondary to the structural finding: Rust achieves a $530\text{--}598\times$ end-to-end speedup over the Python baseline at the peak-scaling cell ($t = 16$) across three independent dataset scales. This end-to-end ratio is dominated by Rust avoiding Python’s result-dict construction (§4.13.1). The search phase itself scales from $22.7\times$ on the smallest calibration dataset to $1.34\times$ on the largest, bending the curve toward the shared hardware ceilings. All these speedups preserve precision = recall = 1.000. The speedup depends on the regime, since negative composability makes the best composition for one regime, regress at another.

6.2 Contributions

The thesis makes five primary contributions.

1. **Engineering re-implementation.** Rust port of Fainder’s query execution pipeline, validated at precision = recall = 1.000. Integrated into the existing Python API via PyO3, delivering a speedup across three independent dataset scales.
2. **Four-ceiling characterisation.** Identification of four performance ceilings mentioned above, on the Sapphire Rapids 8468H micro-architecture, and the regimes under which each ceiling binds.
3. **Negative-composability pattern.** Five independent instances of composability where a second optimisation regresses at scale. An ablation that would compose positively would falsify this pattern.
4. **Pre-flight ceiling-identification.** A probe template diagnoses the binding ceiling at the *target composition* before committing to implementation. When multiple stacked ceilings exist, the ablation methodology (implement, measure, accept) is methodologically insufficient.
5. **Dispatch-on-regime.** Automatic policy that selects build features and deployment flags per workload regime. Valid even across out-of-distribution scenarios.

Contributions (1) and (5) are engineering artefacts, while (2), (3), and (4) are scientific and methodological findings.

6.3 Limitations

Single micro-architecture. All experiments run on a single Sapphire Rapids 8468H configuration. The ceilings introduced in this thesis are based on this micro-architecture. A different micro-architecture (Genoa, Granite Rapids, Grace) would most likely have different per-ceiling thresholds. The methodological contribution (pre-flight ceiling-identification) generalises across micro-architectures, the specific ceiling thresholds and dispatch policy entries do not.

Single-machine evaluation. All experiments run on shared memory. This might introduce a fifth ceiling of Network latency, in case of sharing the index across multiple machines. Asynchronous I/O and work offloading become candidate optimisations in that case.

Density-range scope. The dispatch-on-regime policy is validated at $n_{\text{hists}}/n_{\text{clusters}}$ densities of $\sim 8,200$ (c1024_56gb) and $\sim 26,300$ (c256_56gb). The policy is not validated at densities below $\sim 8,200$ (c4096_56gb) or above $\sim 26,300$ (c128_56gb), both outside the validated range. The policy’s density axis is therefore a prediction rather than a validated entry.

6.4 Future Work

Future work includes three experiments that would extend the structural findings.

(F1) Transparent Huge Pages probe across all builds. The §4.10 mechanism localised the local-ids regression. The $\text{bw}=13$ decoder’s access pattern over the packed buffer drives page-walk pressure. With Transparent Huge Pages enabled (2 MiB pages instead of 4 KiB), TLB-miss frequency drops by $512\times$ per covered page, and the page-walker’s ~ 100 -cycle penalty per miss correspondingly shrinks.

(F2) NUMA-aware build for $t > 48$. §4.6 identified cross-socket UPI as the fourth ceiling and showed that single-socket placement saves 11–13% at $t \leq 48$. Single-socket placement is infeasible at $t > 48$ due to socket 0 having only 48 physical cores. A NUMA-aware build would shard the cluster set across the two sockets at index-load time and pin Rayon workers to physical cores within their NUMA node. Queries would then iterate over node-local clusters preferentially.

(F3) Smaller follow-ons. *Adaptive precision selection.* Optimal f16 vs f32 precision depends on the regime. F32 wins at low t , while f16 wins at high t . An



adaptive scheme at runtime that selects precision based on the thread count (or L3-miss rate) would provide the best of both.

Column-centric engine for small datasets. Column-centric outperformed row-centric at $t = 1$ by $2.17\text{--}2.63\times$ (§4.8.1). Broadening its win region would require addressing the cross-cluster load imbalance that causes columnar to lose by a factor of 2 at $t \geq 8$.

6.5 Closing Remark

The engineering speedups produced in this thesis are specific to this codebase, micro-architecture, and workload. The structural finding is what generalises. On hardware-near workloads the cost of regression for optimisation B on an already applied optimisation A depends on which ceiling has been collected already, making it invisible to per-operator cost models. This makes our finding clear: identify the binding ceiling at the target composition before committing the implementation cost of the next ablation.

Bibliography

- [1] Abadi, D., Boncz, P., Harizopoulos, S., Idreos, S., Madden, S.: The Design and Implementation of Modern Column-Oriented Database Systems. Foundations and Trends in Databases, Now Publishers (2013)
- [2] Abadi, D.J., Madden, S.R., Hachem, N.: Column-stores vs. row-stores: How different are they really? Proceedings of the ACM SIGMOD International Conference on Management of Data pp. 967–980 (2008)
- [3] Acar, U.A., Charguéraud, A., Rainey, M.: Heartbeat scheduling: Provable efficiency for nested parallelism. In: Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018). pp. 769–782 (2018)
- [4] Apache Software Foundation: Apache arrow: A cross-language development platform for in-memory analytics. <https://arrow.apache.org> (2016)
- [5] Asudeh, A., Jin, Z., Jagadish, H. V.: Towards data-centric algorithmic fairness: Challenges, opportunities, and the road ahead. In: Proceedings of the IEEE International Conference on Data Engineering (ICDE) (2022)
- [6] Avnur, R., Hellerstein, J.M.: Eddies: Continuously adaptive query processing. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. pp. 261–272 (2000)
- [7] Bandle, M., Giceva, J., Neumann, T.: To partition, or not to partition, that is the join question in a real system. Proceedings of the VLDB Endowment 14(11), 2101–2113 (2021)
- [8] Baresel, K., Eube, C., Knorr, D., Lutter, L., de Nys, J., et al.: KI-Gebrauch im Studienkontext dokumentieren. Tech. rep., Leuphana Universität Lüneburg (2024), <https://pubdata.leuphana.de/server/api/core/bitstreams/f4c7f7f2-3974-410c-a07b-4e545390561e/content>, version 1.0
- [9] Behme, L., Galhotra, S., Beedkar, K., Markl, V.: Fainder: A fast and accurate index for distribution-aware dataset search. Proceedings of the VLDB Endowment 17(11), 3269–3282 (2024)

- [10] Behme, L., Geißler, T., Agrawal, S., Badura, A., Ueber, J., Beedkar, K., Markl, V.: Finding what you're looking for: A distribution-aware dataset search engine in action. In: Proceedings of the ACM SIGMOD International Conference on Management of Data (Demo) (2025)
- [11] Blumofe, R.D., Leiserson, C.E.: Scheduling multithreaded computations by work stealing. *Journal of the ACM* 46(5), 720–748 (1999)
- [12] Boncz, P.A., Zukowski, M., Nes, N.: MonetDB/X100: Hyper-pipelining query execution. *Biennial Conference on Innovative Data Systems Research (CIDR)* pp. 225–237 (2005)
- [13] Brodal, G.S., Fagerberg, R.: On the limits of cache-obliviousness. *ACM Transactions on Algorithms* 2(2), 307–322 (2006)
- [14] Brooks Jr., F.P.: No silver bullet: Essence and accidents of software engineering. *Computer* 20(4), 10–19 (1987)
- [15] Cormode, G., Garofalakis, M., Haas, P.J., Jermaine, C.: Synopses for massive data: Samples, histograms, wavelets, sketches. *Foundations and Trends in Databases* 4(1-3), 1–294 (2011)
- [16] Drepper, U.: What every programmer should know about memory. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf> (2007), red Hat technical report.
- [17] Dunning, T., Ertl, O.: Computing extremely accurate quantiles using t-digests. *arXiv preprint arXiv:1902.04885* (2019)
- [18] Esmailoghli, M., Galhotra, S., Abedjan, Z.: Data discovery in data lakes: Operations, indexes, systems. *Proceedings of the VLDB Endowment* 18(12) (2025)
- [19] Esmailpour, H., Galhotra, S., Raychaudhury, R., Sintos, S.: A theoretical framework for distribution-aware dataset search. In: *Proceedings of the ACM Symposium on Principles of Database Systems (PODS)* (2025)
- [20] Feng, Z., Lo, E., Kao, B., Xu, W.: ByteSlice: Pushing the envelop of main memory data processing with a new storage layout. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*. pp. 31–46 (2015)
- [21] Ferragina, P., Vinciguerra, G.: The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proceedings of the VLDB Endowment* 13(8), 1162–1175 (2020)

- [22] Fog, A.: The microarchitecture of Intel, AMD and VIA CPUs: An optimization guide for assembly programmers and compiler makers. <https://www.agner.org/optimize/microarchitecture.pdf> (2023), technical University of Denmark.
- [23] Fried, J., Ruan, Z., Ousterhout, A., Belay, A.: Caladan: Mitigating interference at microsecond timescales. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). pp. 281–297 (2020)
- [24] Graham, R.L.: Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics* 17(2), 416–429 (1969)
- [25] Halevy, A., Korn, F., Noy, N.F., Olston, C., Polyzotis, N., Roy, S., Whang, S.E.: Goods: Organizing Google’s datasets. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 795–806 (2016)
- [26] Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., Oliphant, T.E.: Array programming with NumPy. *Nature* 585, 357–362 (2020)
- [27] Hennessy, J.L., Patterson, D.A.: Computer Architecture: A Quantitative Approach. Morgan Kaufmann, 6th edn. (2017)
- [28] Hulsebos, M., Deng, Q., Investigators, T.G.: GitTables: A large-scale corpus of relational tables. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 1–14 (2023)
- [29] Intel Corporation: Intel 64 and IA-32 architectures optimization reference manual. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html> (2023)
- [30] Intel Corporation: Intel intrinsics guide. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/> (2024)
- [31] Ioannidis, Y.: The history of histograms (abridged). In: Proceedings of the 29th International Conference on Very Large Data Bases (VLDB). pp. 19–30 (2003)

- [32] Kersten, T., Leis, V., Kemper, A., Neumann, T., Pavlo, A., Boncz, P.: Everything you always wanted to know about compiled and vectorized queries but were afraid to ask. In: Proceedings of the VLDB Endowment. vol. 11, pp. 2209–2222 (2018)
- [33] Khuong, P.V., Morin, P.: Array layouts for comparison-based searching. ACM Journal of Experimental Algorithmics 22, 1.3:1–1.3:39 (2017)
- [34] Kim, C., Chhugani, J., Satish, N., Sedlar, E., Nguyen, A.D., Kaldewey, T., Lee, V.W., Brandt, S.A., Dubey, P.: FAST: Fast architecture sensitive tree search on modern CPUs and GPUs. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 339–350 (2010)
- [35] Kocberber, O., Grot, B., Picorel, J., Falsafi, B., Lim, K., Ranganathan, P.: Meet the walkers: Accelerating index traversals for in-memory databases. In: Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). pp. 468–479 (2013)
- [36] Kraska, T., Beutel, A., Chi, E.H., Dean, J., Polyzotis, N.: The case for learned index structures. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 489–504 (2018)
- [37] Lameter, C.: NUMA (non-uniform memory access): An overview. Communications of the ACM 56(9), 59–65 (2013)
- [38] Lang, H., Mühlbauer, T., Funke, F., Boncz, P.A., Neumann, T., Kemper, A.: Make the most out of your SIMD investments: Counter control flow divergence in compiled query pipelines. The VLDB Journal 29(2-3), 757–774 (2020)
- [39] Leis, V., Boncz, P., Kemper, A., Neumann, T.: Morsel-driven parallelism: A NUMA-aware query evaluation framework for the many-core age. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 743–754 (2014)
- [40] Leis, V., Kemper, A., Neumann, T.: The adaptive radix tree: ARTful indexing for main-memory databases. In: Proceedings of the 29th IEEE International Conference on Data Engineering (ICDE). pp. 38–49 (2013)
- [41] Li, Y., Patel, J.M.: BitWeaving: Fast scans for main memory data processing. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 289–300 (2013)

- [42] Mahdi Esmailoghli, Jorge-Arnulfo Quiané-Ruiz, Z.A.: Cocoa: Correlation coefficient-aware data augmentation (2021)
- [43] Manegold, S., Boncz, P.A., Kersten, M.L.: Optimizing database architecture for the new bottleneck: Memory access. vol. 9, pp. 266–277 (2000)
- [44] Matsakis, N.D., Klock, F.S.: The Rust language: Memory safety without garbage collection. *Communications of the ACM* 57(4), 14 (2014)
- [45] Maturin Contributors: Maturin: Build and publish Rust crates as Python extensions. <https://www.maturin.rs> (2024)
- [46] McCurdy, C., Vetter, J.: Memphis: Finding and fixing NUMA-related performance problems on multi-core platforms. In: *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. pp. 87–96 (2010)
- [47] Neumann, T.: Efficiently compiling efficient query plans for modern hardware. In: *Proceedings of the VLDB Endowment*. vol. 4, pp. 539–550 (2011)
- [48] Polychroniou, O., Raghavan, A., Ross, K.A.: Rethinking SIMD vectorization for in-memory databases. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data* (2015)
- [49] PyO3 Contributors: PyO3: Rust bindings for Python. <https://pyo3.rs> (2024)
- [50] Python Software Foundation: The global interpreter lock (GIL). <https://wiki.python.org/moin/GlobalInterpreterLock> (2024)
- [51] Raasveldt, M., Mühleisen, H.: DuckDB: an embeddable analytical database. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*. pp. 1981–1984 (2019)
- [52] Satterthwaite, F.E.: An approximate distribution of estimates of variance components. *Biometrics Bulletin* 2(6), 110–114 (1946)
- [53] Schlegel, B., Gemulla, R., Lehner, W.: K-Ary search on modern processors. In: *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN)*. pp. 52–60 (2009)
- [54] Selinger, P.G., Astrahan, M.M., Chamberlin, D.D., Lorie, R.A., Price, T.G.: Access path selection in a relational database management system. In: *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. pp. 23–34 (1979)

- [55] Stone, J., Matsakis, N.D.: Rayon: A data-parallelism library for Rust. <https://github.com/rayon-rs/rayon> (2015)
- [56] Tene, G.: HdrHistogram: A high dynamic range histogram. <https://hdrhistogram.github.io/HdrHistogram/> (2014)
- [57] Tomasulo, R.M.: An efficient algorithm for exploiting multiple arithmetic units. *IBM Journal of Research and Development* 11(1), 25–33 (1967)
- [58] Traub, J., Grulich, P.M., Cuéllar, A.R., Bress, S., Katsifodimos, A., Rabl, T., Markl, V.: Scotty: General and efficient open-source window aggregation for stream processing systems (2020), https://www.redaktion.tu-berlin.de/fileadmin/fg131/Publikation/Papers/Traub_TODS-21-Scotty_preprint.pdf
- [59] Vink, R.: Polars: Fast DataFrame library for Rust and Python. <https://pola.rs> (2020)
- [60] Wallewein-Eising, M., Broneske, D., Saake, G.: SIMD acceleration for main-memory index structures – a survey. In: *Beyond Databases, Architectures and Structures (BDAS)*. pp. 105–119 (2018)
- [61] Weaver, V.M.: Linux perf_event features and overhead. In: *International Workshop on Performance Analysis of Workload Optimized Systems (FastPath)* (2013)
- [62] Welch, B.L.: The generalisation of Student’s problem when several different population variances are involved. *Biometrika* 34(1–2), 28–35 (1947)
- [63] Willhalm, T., Popovici, N., Boshmaf, Y., Plattner, H., Zeier, A., Schaffner, J.: SIMD-scan: Ultra fast in-memory table scan using on-chip vector processing units. *Proceedings of the VLDB Endowment* 2(1), 385–394 (2009)
- [64] Wulf, W.A., McKee, S.A.: Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News* 23(1), 20–24 (1995)
- [65] Yasin, A.: A top-down method for performance analysis and counters architecture. In: *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. pp. 35–44 (2014)
- [66] Yeh, T.Y., Patt, Y.N.: Alternative implementations of two-level adaptive branch prediction. In: *Proceedings of the 19th Annual International Symposium on Computer Architecture (ISCA)*. pp. 124–134 (1992)

- [67] Zhang, Y., Ives, Z.G.: Finding related tables in data lakes for interactive data science. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 1951–1966 (2020)
- [68] Zhou, J., Ross, K.A.: Implementing database operations using SIMD instructions. In: Proceedings of the ACM SIGMOD International Conference on Management of Data. pp. 145–156 (2002)

Appendix A. Reproduction and Implementation Details

This appendix documents the hardware, software, and build configuration used for every measurement in Chapter 4, and provides explicit reproduction recipes for each ablation.

A.1 Hardware

- **Server:** single shared-memory node (192 logical threads total).
- **CPU:** $2 \times$ Intel Xeon Platinum 8468H (Sapphire Rapids, 48 physical cores per socket, hyperthreading enabled, up to 2.1 GHz base / 3.8 GHz turbo).
- **Caches:** 48 KB L1-dcache per core; 2 MB L2 per core; 105 MB L3 per socket (shared within a NUMA node).
- **NUMA:** two nodes, one per socket. Node 0 holds logical CPUs 0–47 and their hyperthread siblings 96–143; node 1 holds 48–95 and 144–191. The 48-physical-core-per-node bound is the reason thread counts $t \geq 64$ span both NUMA nodes.
- **Memory:** ≈ 1 TB DDR5 across both sockets, eight channels per socket.
- **Storage:** NVMe SSD (dataset loads); `/dev/shm` RAM-backed tmpfs (storage-hierarchy ablation).

A.2 Software

- **OS:** Ubuntu 22.04.5 LTS, Linux kernel 5.15.0-176-generic, glibc 2.35.
- **Rust:** `rustc 1.94.1` (LLVM 21.1.8). Cargo default `release` profile (`opt-level=3`); `-C target-cpu=native` supplied via `.cargo/config.toml`.
- **Python:** 3.10.12; NumPy 1.26.4; zstandard 0.22.0.



- **Rust crates** (Cargo.lock resolved): pyo3 0.22.6; numpy 0.22.1; rayon 1.11.0; ndarray 0.16.1; half 2.7.1; mimalloc 0.1.50; crossbeam-deque 0.8.6; thiserror 1.0.69.
- **Build tool:** Maturin 1.7. The Rust extension is compiled via `maturin develop --release [--features ...]` and installed into the active virtualenv.
- **Profiling:** Linux perf 5.15 (/proc/sys/kernel/perf_event_paranoid = 1).

A.3 Repository Structure

The full repository layout is documented in the artefact's README (<https://github.com/tmukh/Fainder-Redone>). Rust engine lives under `src/`, the Python integration under `fainder/`, the benchmark drivers under `scripts/`, the figure-generation code under `analysis/`, and the raw measurement database under `logs/`.

A.4 Feature-Flag Matrix

The Rust crate uses Cargo feature flags to compile alternative implementations of `SubIndex`, the search loop, and the parallelism strategy. Exactly one feature combination is active per build.



Flag	Default	Effect
(none)	yes	SoA layout, f32 values, stdlib <code>partition_point</code> , query-level parallelism
<code>aos</code>	no	Array-of-Structs layout: <code>Vec<(f32,u32)></code>
<code>f16</code>	no	Half-precision percentile values
<code>cluster-par</code>	no	Nested <code>par_iter</code> over clusters (documented negative result)
<code>simd</code>	no	AVX-512 leaf-stage compare in the binary-search inner loop
<code>batch-search</code>	no	8-way interleaved binary search (columnar engine only)
<code>horizontal-simd</code>	no	AVX-512 16-query horizontal lockstep
<code>pgm</code>	no	PGM learned-index search
<code>packed-ids</code>	no	Bit-packed per-cluster identifiers
<code>local-ids, -bench, -noalloc</code>	no	Reindexed 13-bit local IDs (three mutually-exclusive variants)
<code>pooled</code>	no	Pre-allocated per-query output buffer
<code>pin-cores</code>	no	Rayon workers pinned to distinct physical cores
<code>cluster-prefetch</code>	no	Software <code>mm_prefetch</code> of cluster $c+1$
<code>mimalloc</code>	no	mimalloc allocator
<code>query-batch</code>	no	Column-centric inner loop over K -query batches
<code>morsel</code>	no	Chase-Lev deque phase-aligned scheduler

The `eytzinger` and `kary` feature flags belong to an earlier campaign build and are not present in the final integrated build’s `Cargo.toml`.

A.5 Reproduction Recipes

All commands assume the working directory is the repository root and the Python virtualenv is activated (`source venv/bin/activate`).

Dataset naming. The shell scripts accept legacy datasets (`eval_10gb`, `eval_medium`, `dev_small`) as CLI arguments, matching the on-disk dataset folder names on the measurement server. These map to the naming used throughout the thesis body and the measurement database as follows:

Script argument	Thesis name	Notes
<code>eval_10gb</code>	<code>c256_10gb</code>	$K = 256$, ~10 GB
<code>eval_medium</code>	<code>c256_30gb</code>	$K = 256$, ~30 GB
(built fresh)	<code>c256_56gb</code>	$K = 256$, ~56 GB, full workload
(built fresh)	<code>c1024_56gb</code>	$K = 1024$; reclusters <code>c256_56gb</code> for the out-of-distribution probe
<code>dev_small</code>	—	Smoke test only; not used in results

Build. The default (SoA + f32 + `partition_point`) build is:

```
maturin develop --release
```




For an ablation-specific build, pass the feature flag:

```
maturin develop --release --features f16
maturin develop --release \
    --features "pooled f16 simd pin-cores cluster-prefetch mimalloc"
```

Ablation runs. Each ablation has a shell driver that (1) compiles the relevant build, (2) runs the thread sweep with `--suppress-results`, (3) extracts the Rust `index`-based query execution time per log, and (4) prints a summary table. Representative commands:

```
# Thread sweep (default build, 7 thread counts 1-64)
bash scripts/ablation_parallel.sh eval_medium
```

```
# f16 vs f32 (rebuilds twice)
bash scripts/ablation_f16.sh eval_medium
```

```
# Columnar vs row-centric
bash scripts/ablation_columnar.sh eval_medium
```

```
# NUMA pinning
bash scripts/ablation_numa.sh eval_medium
```

Perf counters. The perf-stat runs use `--delay=40000` to skip the ~ 35 s of load+init+warmup, and repeat the measured query set five times to avoid under-sampling:

```
bash scripts/perf_comprehensive.sh eval_medium
bash scripts/perf_branch_misses.sh
```

Figure regeneration. Figures are regenerated from the measurement logs without re-running the experiments:

```
python analysis/plot_all_results.py
python analysis/plot_hardware_ablation.py
```

A.6 Query Metric

All ablation runs use `suppress_results=True`, which returns empty result vectors while still executing the full binary-search work. This isolates search time from the Python set-building phase that dominated pre-PyO3-fix measurements and would otherwise swamp engineering-scale differences between variants. The extracted metric is the log line emitted by the Rust backend:

```
Rust index-based query execution time: X.XXXs
```

This represents the wall-clock time of the Rayon-parallel query phase only, excluding the one-time `FainderIndex` construction from NumPy arrays (~ 17 s on `eval_medium`) and the initial index load (0.013 s via `mmap`).